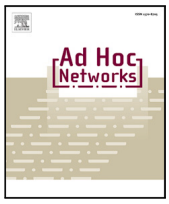
Ad Hoc Networks 175 (2025) 103871

Contents lists available at ScienceDirect

Ad Hoc Networks

journal homepage: www.elsevier.com/locate/adhoc



SC-MLIDS: Fusion-based Machine Learning Framework for Intrusion Detection in Wireless Sensor Networks

Hongwei Zhang [a], Darshana Upadhyay [a,*], Marzia Zaman [b], Achin Jain [c], Srinivas Sampalli [a]

[a] Faculty of Computer Science, Dalhousie University, Halifax, B3H 4R2, NS, Canada
[b] Research and Development Department, Cistel Technology Inc., Ottawa, K2E 7V7, ON, Canada
[c] Research and Development Department, Norleaf Networks, Gatineau, J8Y 2V5, QC, Canada

ARTICLE INFO

Keywords:
Wireless sensor networks
Machine learning
Intrusion detection systems
Ensemble learning
Federated learning
Server–client models
Model aggregation
Network security
Fusion algorithms

ABSTRACT

This paper proposes the Server–Client Machine Learning Intrusion Detection System (SC-MLIDS), a novel fusion framework designed to enhance security in Wireless Sensor Networks (WSNs), which are inherently vulnerable to various security threats due to their distributed nature and resource constraints. Traditional Intrusion Detection Systems (IDSs) often face challenges with high computational demands and privacy issues. SC-MLIDS addresses these problems by integrating Federated Learning (FL) with a multi-sensor fusion approach to implementing two layers of defence that operate independently of specific attack types. Moreover, this framework leverages a server–client architecture to efficiently manage and process data from sensor nodes, sink nodes, and gateways within the network. The core innovation of SC-MLIDS lies in its dual model aggregation algorithms at the gateway: one assesses model performance and weight, while the other uses majority voting to integrate predictions from both client and server models. As a result, this approach reduces redundant data transmissions and enhances detection accuracy, making it more effective than conventional methods in WSNs. Our proposed framework outperforms current state-of-the-art techniques, achieving F1-scores of 99.78% and 98.80% for the two aggregation algorithms, namely, Weighted Score and Majority Voting. This validation demonstrates the effectiveness of SC-MLIDS in providing accurate intrusion detection and robust data management.

1. Introduction

The continuous advancement of Wireless Sensor Networks (WSNs) and their associated technologies has led to widespread applications across various industrial sectors, enhancing productivity through intelligent monitoring and management [1,2]. However, the distributed and resource-constrained nature of WSNs exposes them to numerous vulnerabilities. Researchers have proposed various Intrusion Detection Systems (IDSs) to identify suspicious activities, including both anomaly and misuse detection [3].

The emergence of Machine Learning (ML) has significantly expanded the capabilities of IDSs by offering enhanced real-time detection features. ML-driven approaches have shown improved performance in detecting network attacks through the integration of data from diverse sources, making them a central focus in network security research. ML enables the development of models trained on large-scale, multi-source datasets, facilitating accurate, automated, and adaptive risk assessment [4].

Traditionally, intrusion detection in WSNs has utilized methodologies such as ML-based systems, and hybrid approaches that combine anomaly detection with signature-based methods. These systems typically operate at the sensor node level or within clustered architectures using various techniques, such as autonomous detection agents, predefined rules, or hierarchical models [5–8].

Despite these advancements, several research gaps persist. These include increased node workload, limited adaptability to complex attacks, and inefficiencies in data management. For example, some methods rely on static detection rules that cannot dynamically adjust to new threats, while others depend heavily on predefined signatures, reducing their effectiveness against novel attacks. Many approaches are also constrained by their architecture, failing to incorporate model ensembles or address redundant data transmission. Furthermore, most existing ML-based IDSs are tailored to detect specific types of network attacks and rely on limited data sources, leading to constrained solutions. Given

* Corresponding author.
E-mail addresses: hongweizhang@dal.ca (H. Zhang), darshana@dal.ca (D. Upadhyay), marzia@cistel.com (M. Zaman), ajain@norleaf.ca (A. Jain), srini@cs.dal.ca (S. Sampalli).

https://doi.org/10.1016/j.adhoc.2025.103871
Received 18 October 2024; Received in revised form 22 February 2025; Accepted 12 April 2025
Available online 26 April 2025

Table 1
Comparison with our earlier work.

Aspect	Early Work (Hybrid ML-based IDS) [9]	Current Work (SC-MLIDS)	Key Improvement
Architecture	Standard WSN architecture with distributed intrusion detection	Server–client architecture with federated learning and multi-sensor fusion integration	Multi-layer coordination reduces computational burden on sensors; supports diverse WSN architectures
Data Handling	Analyzed sensing data (sensor level) and network traffic (gateway level)	Integrates multi-sensor application-level data and network traffic data for comprehensive detection	Supports diverse data sources and types (multimodal approach)
Machine Learning Models	Random forest models trained separately for sensor nodes and the gateway	No restrictions on ML algorithms; supports homogeneous/heterogeneous models for fusion	Flexibility in model selection improves adaptability to different scenarios
Fusion Algorithms	Aggregation prediction algorithm for combining results	Improved aggregation algorithm and introduced voting algorithm to accommodate varying demands	Improved prediction accuracy and reliability through algorithm improvement and addition
Computational Load	Distributed computation across sensors and the gateway	Server–client design offloads processing to the server; reduces sensor node workload	Minimizes redundant data transmission and improves energy efficiency
Adaptability	Limited to specific WSN architectures	Structural adjustments allowed to align with any WSN architecture and application-specific needs	Addresses practical deployment challenges across diverse geographic/environmental conditions
Flexibility	Fixed two-layer detection (sensor + gateway)	Supports unlimited ML models and multimodal data fusion	Accommodates evolving intrusion detection requirements and architecture extensions
Performance	Achieved 99% accuracy with high performance in standard metrics	Enhanced accuracy and reliability through advanced fusion techniques	Balances high accuracy with real-world practicality and scalability

the dynamic, multi-source nature of WSN environments, there is a pressing need for an adaptable and comprehensive IDS framework that integrates data fusion across multiple sensors and processes.

In our earlier work [9], we introduced a hybrid ML-based IDS for WSNs, built upon the standard WSN architecture. To address resource constraints, the intrusion detection process was distributed, with different types of data being analyzed at both the sensor level and the gateway level. This approach improved accuracy and comprehensiveness in intrusion detection by optimizing the computation of prediction results.

The IDS is designed to perform two-layer intrusion detection by identifying anomalies in sensing data at the sensor node level and network traffic anomalies at the gateway. We utilized random forests to train individual models for each of the three sink nodes and one gateway to facilitate intrusion detection. In our experiments, we present the results of each model and the aggregation prediction algorithm, both before and after applying the IDS. The system achieved 99% accuracy along with high performance across the other three evaluation metrics.

However, this IDS has several limitations, particularly regarding its practical applications. Over decades of development, WSNs have been widely adopted across various fields, requiring deployment to be tailored to specific application scenarios. Consequently, the architecture of WSNs varies significantly depending on the field, application context, and geographic environment. As a result, IDS research for WSNs, including our earlier work, must address these challenges that impede practical implementation.

In this paper, we extend our previous research by introducing the concept of multimodality to overcome its limitations and meet the diverse requirements of various WSN architectures and application scenarios. This enhancement significantly improves the practicality, adaptability, and flexibility of the IDS, as summarized in Table 1. Specifically, we propose a novel fusion framework, the Server–Client Machine Learning Intrusion Detection System (SC-MLIDS), which integrates Federated Learning (FL) with multi-sensor fusion techniques. SC-MLIDS builds upon earlier work by introducing advancements in data types, detection models, fusion algorithms, and architectural design.

SC-MLIDS supports the integration of diverse data sources, such as multi-sensor application-level data and network traffic data, enabling comprehensive intrusion detection. The framework leverages a multimodal approach, and imposes no restrictions on the algorithms or the number of ML models utilized, accommodating both homogeneous and heterogeneous base models to ensemble. SC-MLIDS incorporates an innovative voting algorithm to enhance the effectiveness of model fusion, improving the accuracy and reliability of predictions. Furthermore, its design allows for structural adjustments to align with different WSN architectures and application-specific requirements, ensuring its adaptability across various scenarios. By implementing these enhancements, SC-MLIDS achieves a high degree of flexibility, adaptability, and accuracy, addressing the complex and evolving demands of intrusion detection in WSNs.

In SC-MLIDS, the sink nodes of the WSN serve as clients to locally train the models, while the gateway serves as a server to receive model outputs from multiple clients and integrate the prediction results through two proposed aggregation prediction algorithms (Weighted Score and Majority Voting). In summary, the SC-MLIDS framework adopts a server–client architecture to manage and process data, significantly reducing the computational burden on sensor nodes. Its dual-model aggregation algorithms improve detection accuracy while dynamically adapting to various attack types. By minimizing redundant data transmissions and employing advanced evaluation and fusion techniques, SC-MLIDS provides a comprehensive, efficient, and highly accurate solution for intrusion detection in WSNs.

1.1. Major contributions

The contributions of this paper are summarized as follows:

1. We propose a fusion-based SC-MLIDS framework that integrates server and client components corresponding to the gateway and sink nodes in WSNs. This framework employs a two-layer validation process for both sensing and network traffic data. By leveraging multi-source data fusion, we aim to provide extensive and adaptable detection capabilities across diverse WSN environments.
2. We utilize two aggregation prediction algorithms in our proposed framework, namely, the Weighted Score algorithm and the Majority Voting algorithm. These algorithms enhance intrusion detection by merging predictions from multiple ML models. Specifically, the Weighted Score algorithm assesses model performance based on metrics and weights, while the Majority Voting algorithm combines predictions. Through this integration at the gateway, we aim to ensure robust and accurate detection by leveraging diverse model outputs.

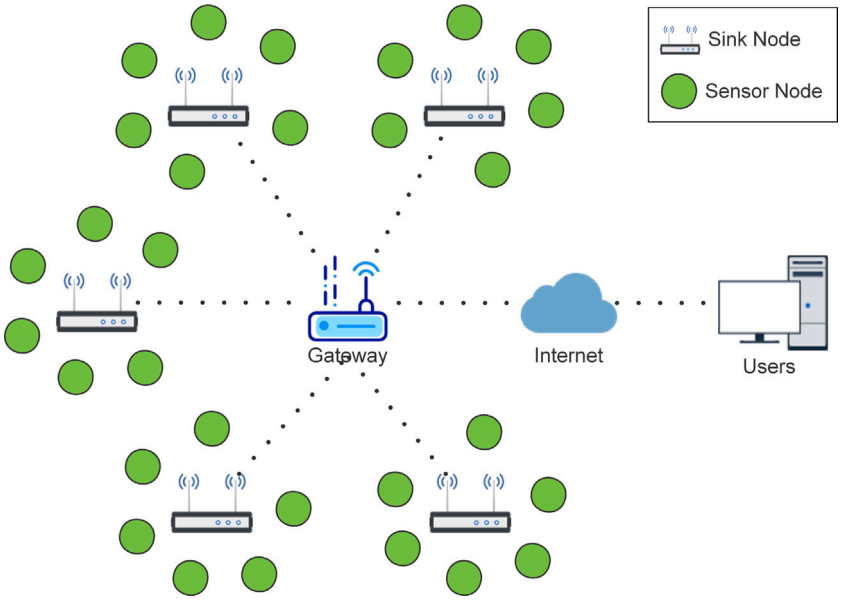


Fig. 1. WSNs architecture.

3. We integrate application/sensor data and network traffic data to generate a more diverse and realistic dataset. This combination supports the development of more effective models using FL with ensemble techniques. Through this approach, we aim to improve computational speed via distributed computing and enhance security by preventing the transfer of application data to the server.
4. We evaluate the proposed framework through simulation experiments, considering various performance metrics, namely, Accuracy, Precision, Recall, and F1-score, for the two aggregation algorithms to ensure effective assessment. Furthermore, we compare our results with current state-of-the-art techniques.

1.2. Outline of the paper

The remainder of this paper is structured as follows: Section 2 provides a comprehensive background and reviews related works. In Section 3, we detail the design of the proposed framework and algorithms. Section 4 presents the dataset, implementation details, design of the experiment and results. Section 5 discusses the experimental results and highlights the associated limitations. The paper concludes with Section 6, which summarizes the key findings, emphasizes the main contributions of our work, and outlines potential directions for future research.

2. Background and related works

2.1. Wireless sensor networks

Wireless Sensor Networks (WSNs) are self-configuring networks that contain many wireless sensor nodes [10,11]. These nodes are used for sensing and monitoring the environment in which they are deployed, and for collecting data. The WSN architecture, as shown in Fig. 1, consists of the following components, namely, sensor nodes, sink nodes and the gateway [12].

Sensor nodes are tiny devices with low power consumption, integrated with sensors, microcontroller, transceiver, power supply, and operating system [13,14]. The sensor node senses and collects information from the environment through various types of equipped sensors. The required sensor types and data types vary depending on the application scenario. Common sensing data include temperature, humidity, pressure, light, and sound [15].

The microcontroller, acting as the CPU of the sensor node, controls all hardware components by running a micro-operating system [16,17]. It performs the collection and simple sensing data processing, then sends it to the sink node via the transceiver, or communicates with other sensor nodes [13].

The sink node is the data collection unit [18], responsible for receiving and organizing the data sent from the wireless sensor nodes in the covered area. In WSN architectures, sink nodes often communicate with the gateway or the Internet to upload data received from sensor nodes [18,19]. However, in certain WSN configurations, the gateway may be replaced by the sink node, allowing the sensor nodes to upload data directly via the sink node [20]. Such arrangements enhance the efficiency of data transmission. In more extensive WSN architectures, which include a large number of sensor nodes, the deployment of multiple sink nodes can further enhance data collection efficiency [19,21]. By assigning each sink node responsibility for only a limited number of sensor nodes to communicate and process data, it is possible to avoid issues such as network latency and throughput limitations that arise when a single sink node is overwhelmed by communications from numerous sensor nodes [21].

The gateway receives the data generated by the sensor nodes, which is transmitted via the sink node, and uploads it to the Internet [22]. This process enables users to access, process, and analyze the data. The superior hardware configuration of the gateway facilitates the implementation of functions that are not feasible with sensor nodes due to various limitations. An example of such a function is security solutions to counter network attacks in WSNs [23].

Advances in technologies related to WSNs have led to their rapid application across various industries. In the military field, WSNs can provide services such as data collection, communication, and battlefield surveillance [24]. In the healthcare sector, WSNs enable continuous remote monitoring of the health statuses of patients without the constraints of fixed locations, resulting in improved quality of healthcare services [25]. WSNs facilitate intelligence in the agricultural field by monitoring farms, monitoring temperature changes, and managing irrigation systems, thereby increasing agricultural productivity while reducing costs [26]. In the smart home sector, WSNs provide users with enhanced home security through monitoring of temperature, smoke, and gas leakages [27]. Additionally, WSNs offer cost-effective solutions to societal challenges, such as forest fire detection and air quality monitoring [28,29].

WSNs are characterized by their small node size, low cost, and the capacity for widespread deployment. However, these features also result in several limitations, including computational power, power consumption, power supply, communication and sensing capabilities [30,31]. Consequently, these limitations pose multiple challenges for WSNs, particularly in network security. Common attacks on WSNs include Denial of Service (DoS) attacks, Sybil attacks, Blackhole/Sinkhole attacks, Hello Flood attacks, and Wormhole attacks [32].

2.2. Internet of things

Internet of Things (IoT) forms a network infrastructure with many sensing, communication, network, and information processing devices [33,34]. This infrastructure facilitates data exchange among devices, between devices and servers, and between devices and the Internet [35]. The fundamental technology of IoT is Radio Frequency Identification (RFID), which allows devices embedded with RFID tags to be read through Near Field Communication (NFC) technology [34,36]. This capability enables unique identification, tracking, and monitoring of the devices [33]. IoT has achieved significant advancement with the introduction of additional technologies such as WSNs, Machine-to-Machine (M2M) communication, and low-power Personal Area Networks (PANs), thereby becoming more intelligent and interconnected [37]. IoT has found extensive applications across various sectors of human society, including intelligent control and monitoring in industry and agriculture, smart cities, smart homes, and healthcare [38–40].

2.3. Federated learning

Federated Learning (FL) is a distributed ML approach that involves model training through the collaboration of multiple clients with a server [41,42]. The process of FL is summarized in six steps [41–43]:

1. A generic initialization model is configured as the global model on the server.
2. This global model is then distributed to all clients.
3. Clients use their local data to train the model, thus generating customized local models.
4. The parameters of the trained model, which differ from those of the global model, are sent from the clients to the server.
5. Upon receiving these parameters, the server aggregates them using a strategy such as Federated Averaging (FedAvg), allowing for the updating of the global model.
6. Repeat steps 4 and 5, with the updated global model's parameters sent back to the clients for subsequent rounds of training. The cycle continues until the model converges.

In FL, the aggregation of local models into a global model is formalized as follows. Let θ_{global} represent the global model parameters and θ_i denote the local model parameters of client i. The FedAvg algorithm updates the global model as below:

$$\theta_{global} \leftarrow \frac{1}{N}\sum_{i=1}^{N}\theta_i \tag{1}$$

where N is the number of clients. Eq. (1) represents the aggregation of parameters of the local model to form the updated global model.

FL is well-suited for IoT applications, due to its decentralized nature, data privacy protection, and resource conservation [44–46]. The rapid expansion of IoT has led to an enormous volume of data generated by many IoT devices, presenting challenges related to data overload, data security, and data privacy [44]. The distributed feature of FL eliminates the need for centralized data processing. Configuring numerous local clients alleviates computational burdens and avoids privacy and security issues associated with data transmission. Therefore, FL offers an effective and privacy-preserving solution for IoT environments.

2.4. Ensemble learning

Ensemble Learning (EL) is a predictive methodology combining multiple ML models. This approach involves training classifiers on the same dataset using different algorithms, thereby generating different models [47].

The ensemble model is expressed mathematically as follows:

$$f_{\text{ensemble}}(x) = \frac{1}{M}\sum_{m=1}^{M} f_m(x) \tag{2}$$

where $f_{\text{ensemble}}(x)$ is the prediction of the ensemble model, M is the number of individual models, and $f_m(x)$ represents the prediction from the m^{th} model. Additionally, for classification, the ensemble approach generally uses a weighted majority vote as depicted in Eq. (3):

$$\hat{y} = \text{argmax}_c\left(\sum_{m=1}^{M} w_m \cdot (f_m(x) = c)\right) \tag{3}$$

where $\hat{y}$ is the final predicted class, w_m is the weight assigned to the m^{th} model, and $(f_m(x) = c)$ is an indicator function that returns 1 if the m^{th} model predicts class c, and 0 otherwise.

The primary objective is to synthesize a new model that integrates the strengths of distinct classification algorithms. This integration aims to transcend the limitations inherent in relying on a single classification algorithm, thereby enhancing the performance of predictions [48].

The two aggregation prediction algorithms we propose are based on the concept of bagging in EL. The Weighted Score algorithm calculates the final prediction result by considering both the weights and performance metrics of individual models. The Majority Voting algorithm adopts a model grouping strategy that performs majority voting across the predictions of client models, and then combines the result with the server-side prediction output.

2.5. Related works

Traditional IDS for WSNs are classified into anomaly detection and misuse detection [3]. Anomaly detection systems primarily focus on identifying anomalies in nodes, networks, and data within WSNs [49]. Misuse detection involves the detection of known attacks through the pre-configuration of attack signatures [3]. However, the evolution of ML techniques has introduced more possibilities for IDS in WSNs. ML-based IDS for WSNs can effectively identify and filter anomalous data, thereby saving computational and network resources. Furthermore, such systems can autonomously detect, learn from, and prevent various network attacks and vulnerabilities, eliminating the need for human intervention [50].

Current research in ML-based IDS for WSNs primarily concentrates on deploying specific ML algorithms for the identification and detection of particular types of network attacks. For example, studies [50,51] introduce methods using ML algorithms such as the K-Nearest Neighbors (KNN), Decision Trees (DT), and Support Vector Machines (SVM), for outlier detection and the identification of specific network attacks. Concurrently, the advancement of ML has promoted innovative approaches such as Boosting and Deep Learning (DL) based schemes. For instance, research in [52] investigates the comparison of three novel Boosting methods against three traditional ML techniques for detecting network attacks in WSNs. Additionally, [53] explores the use of Deep Neural Networks (DNN) to solve the limitations inherent in traditional ML, especially in response to imbalanced attacks.

Given the potential constraints associated with single ML classifiers, some studies have advocated for model ensemble strategies to enhance detection capabilities. For example, [54] introduces an IDS scheme that integrates Random Forest (RF), Density-based Spatial Clustering of Applications with Noise (DBSCAN), and Restricted Boltzmann Machine (RBM). To enhance detection efficiency, certain methods, such as the one proposed in [55], apply feature selection algorithms to filter

Table 2
Summary of related works.

Authors (Year)	Methodology	Research Gaps
Yu and Tsai (2008) [58]	ML-based IDS using autonomous IDAs on sensor nodes and base station fine-tuning mechanisms.	Intrusion detection is implemented at the sensor node level, increases the workload of the node, alerts rely on user judgment, analysis of local features and adjacent node communications cannot cope with multiple attacks.
Medhat et al. (2015) [5]	DT-based IDS implemented at sensor nodes for defining detection rules through feature selection.	Limited to detecting data anomalies at the sensor node level using DT with predefined rules, without supporting model selection and dynamic adjustment to deal with complex attacks.
Maleh et al. (2015) [6]	Hybrid IDS combining ML anomaly detection and signature-based detection in a clustered WSN.	Restricted WSN architecture, relies on predefined signatures, lacks detection of application data, and does not support model ensemble and adjustment.
Zhang et al. (2020) [7]	Hierarchical IDS using MK-ELM for anomaly detection in clustered WSN architecture.	Relies on a single model, analyzes only application data, lacks network data detection, does not support model selection and fusion, and cannot cope with complex attacks.
Alruhaily and Ibrahim (2021) [8]	Two-layer IDS using NB and RF classifiers for network traffic classification and attack detection.	Limited by WSN architecture, model selection, and attack types, deploying models at the sensor node level increases the workload of the nodes, fails to reduce redundant data transfers in resource-constrained WSN environments, and fails to incorporate the benefits of different models.

key features. This approach is aimed at reducing the time required for attack detection and ensuring efficient identification of attacks. Additionally, research efforts have been directed towards developing detection methods for specific data types. The study in [56] focuses on detecting anomalies in sensing data, while [57] combines traditional ML and DL techniques to identify malicious nodes through network traffic classification.

The aforementioned studies represent the main research direction of ML-based intrusion detection solutions for WSNs, offering valuable insights into WSN security. However, these researches mainly focus on applying specific models or algorithms to address unique network security challenges. There exists a notable gap in their consideration of comprehensive network security issues in real-world WSN application scenarios. The following key papers have made contributions to this research area and provided crucial foundations and inspirations for our SC-MLIDS, as summarized in Table 2.

Yu and Tsai [58] propose an ML-based IDS for WSNs that addresses the limitations of detecting specific types of attacks. In their framework, each sensor node is equipped with an Intrusion Detection Agent (IDA). These IDAs operate autonomously with two primary components: the Local Intrusion Detection Component (LIDC) and the Packet-based Intrusion Detection Component (PIDC). The LIDC analyzes local features to determine if the host node is under attack, while the PIDC monitors the communication activities of neighboring nodes identified as suspicious by the LIDC. Alerts from the IDAs are transmitted to and examined by users through the base station. When false alarms are detected, the base station fine-tunes and optimizes the model, enhancing its accuracy and reliability. This automated feature significantly improves the IDS's adaptability. Medhat et al. [5] developed two DT-based algorithms for defining detection rules in WSNs. These algorithms target two configurations: base station-sink-sensor and sink-sensor architectures. In both setups, the IDS is implemented at the sensor nodes level using supervised or unsupervised learning algorithms to generate detection rules through feature selection. Intrusion detection is achieved by applying DT classifiers that use these rules to detect anomalies in sensing data. These binary DT-based algorithms are notable for their high detection accuracy, achieved with fewer features and lower resource demands. Maleh et al. [6] propose an IDS that combines SVM-based anomaly detection with predefined attack rule signatures, enhancing the system's capability to identify attacks or anomalies. Designed for a clustered WSN topology, it features dynamically elected cluster heads responsible for monitoring and authenticating nodes. This hybrid approach leverages both ML anomaly detection and signature-based IDS, resulting in a lightweight, efficient, and highly accurate solution. Zhang et al. [7] propose a hierarchical IDS using the Extreme Learning Machine (ELM) methodology. In their framework, the cluster head node preprocesses data collected by sensor nodes and sends it to the sink node for feature extraction relevant to intrusion detection. This data is then forwarded to the management node over the Internet. The management node's intrusion detection module uses its Multi-Kernel ELM (MK-ELM) model to detect intrusions. The results are then sent to an anomaly processing module for further analysis and decision-making. Simulations demonstrate that their model achieves high-precision detection while reducing detection time. Alruhaily and Ibrahim [8] propose a two-layer ML defence strategy for IDS in WSNs. At the sensor nodes layer, a Naive Bayes (NB) classifier makes an initial classification of packets as normal or malicious. Malicious traffic is then analyzed in the cloud using an RF multi-class classifier, which identifies specific attack types and guides corresponding defence mechanisms. Their approach efficiently utilizes network resources and demonstrates high accuracy in detecting normal traffic and four distinct attack types.

A review of prior research has highlighted the significant improvements made in hybrid ML-based IDSs for WSNs. However, some critical shortcomings are still present. Current research has focused on the use of specific ML algorithms or the combination of ML algorithms with traditional IDS approaches (e.g., signature rules) to identify specific attack types. Additionally, these studies have notable limitations in terms of ML algorithm selection, model optimization, processing efficiency, resource consumption, and deployment adaptability in WSNs, especially in dynamically changing network environments and diverse security threats. These gaps highlight the importance of the need for a hybrid ML IDS framework that is not only highly flexible and adaptable but also capable of transcending the limitations of attack types.

Proposed approach & novelty: To address these gaps, this paper proposes the Server–Client Machine Learning Intrusion Detection System (SC-MLIDS) framework. The SC-MLIDS framework is designed based on the data characteristics and architectural features of WSNs. By implementing a two-layer detection mechanism, it aims to achieve lightweight intrusion detection that is both highly accurate and efficient. The adoption of a server–client architecture not only ensures that the framework is highly adaptable and scalable but also allows the framework to flexibly adapt to dynamically changing network security threats and requirements. In addition, the framework supports diverse algorithms and model adjustment, which can adapt to different model combinations generated by diverse ML algorithms to meet different WSN architectures and application requirements.

3. Proposed framework

3.1. Framework design

In large WSNs, multiple sink nodes are tasked with data collection from assigned wireless sensor nodes. These networks use a gateway that serves as an intermediary between the sink nodes and the Internet. This gateway's primary functions include communication with each

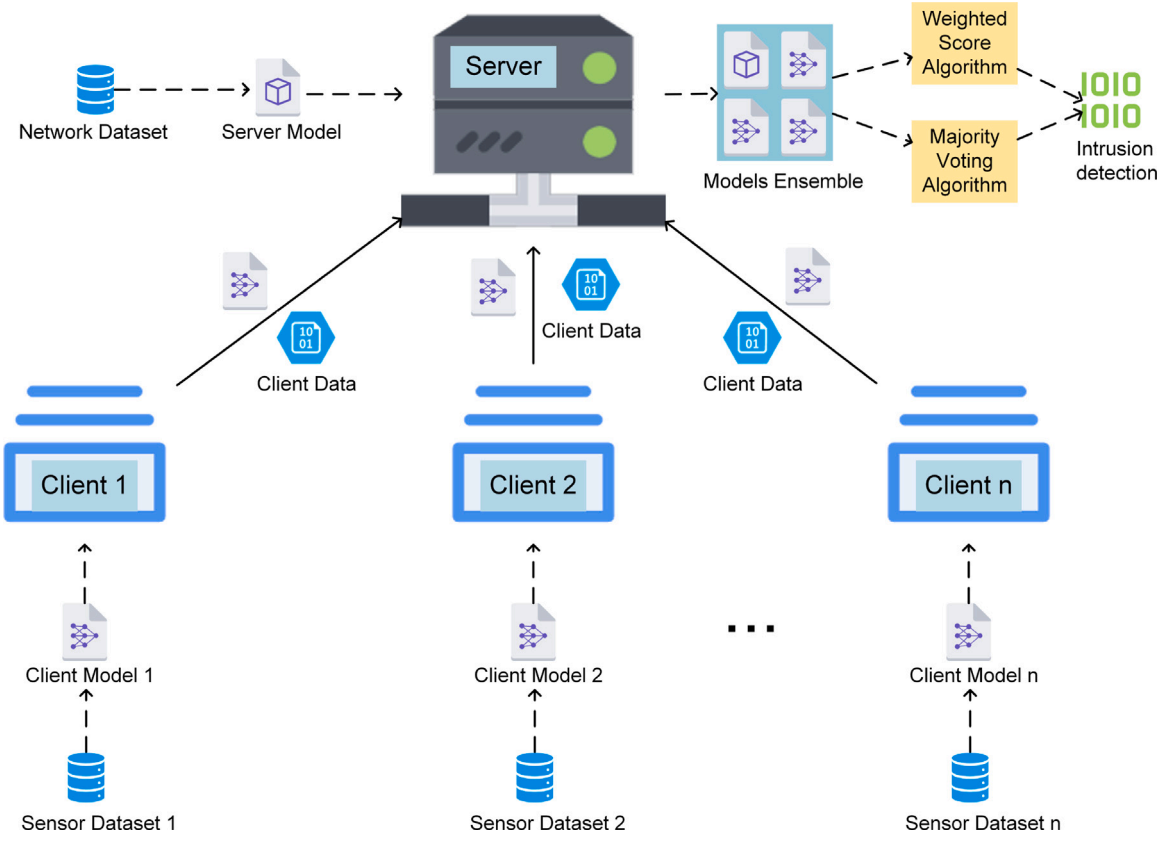


Fig. 2. Proposed SC-MLIDS framework.

sink node and the aggregation and processing of data. This described architecture shares similarities with the global–local model structure in FL, as applied within the IoT context.

Therefore, this paper proposed a framework for WSNs, a hybrid ML IDS with integrated server and client models, as shown in Fig. 2. In the SC-MLIDS framework, both the client models from sink nodes and the server model from the gateway are used for intrusion detection. The gateway integrates two types of models: a server model based on network traffic data and multiple client models based on sensing data. By implementing a two-layer detection mechanism, it aims to achieve lightweight intrusion detection that is both highly accurate and efficient. The SC-MLIDS framework contains two primary components: a server and multiple clients:

- **Client:** Represents the sink nodes in the WSN.
- **Client Model:** This component is responsible for validating sensing data. Such data are generated and transmitted by wireless sensor nodes that are managed by the sink node.
- **Server:** Represents the gateway in the WSN.
- **Server Model:** This component is responsible for validating network traffic data. This network traffic is associated with the sensing data that the gateway receives from the sink node.
- **Models Ensemble:** The model fusion is performed by using the two proposed aggregation algorithms to generate the prediction of the intrusion or not.
- **Client Data:** The sensing data that collected by the sink node from its administered wireless sensor nodes and filtered by the client model.
- **Dataset:** The application/sensing data from the sensor nodes, and the network traffic data generated during data transmission, are used to train both the client and server models.

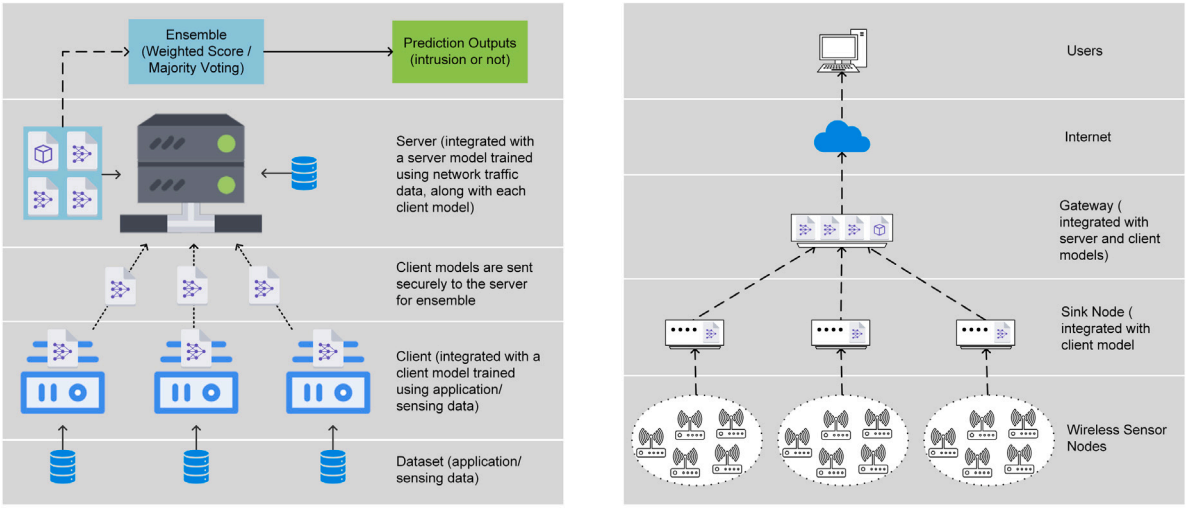
Based on the ML features, this framework can be divided into two phases: the model training and evaluation phase, and the model deployment and operation phase, which performs model preparation and intrusion detection, respectively.

During the model training and evaluation phase, it is crucial to prepare a dataset containing both sensing data and the corresponding network traffic data. This dataset is used for training the ML models on both the clients and the server. The model training and evaluation phase of SC-MLIDS is shown in Fig. 3(a) and can be summarized in seven steps:

1. **Distribute the split sensor dataset:** Each client receives a portion of the split sensor dataset.
2. **Train client models:** In each client, a client model is trained using the assigned sensor dataset.
3. **Compress and encrypt client model files:** After training, the client model files are compressed and encrypted for secure transmission.
4. **Transmit client model files to the server:** These compressed and encrypted client model files are then sent to the server.
5. **Server reception and processing:** Upon receipt, the server decompresses and decrypts all the client model files.
6. **Server model training:** The server trains a server model using the network traffic dataset.
7. **Model ensemble:** The server ensembles all the client and server models using the aggregation prediction algorithms.

The model deployment and operation phase of SC-MLIDS is shown in Fig. 3(b) and can be summarized in six steps:

1. **Data collection by sensor nodes:** These nodes actively sense environmental conditions, generating relevant sensing data.
2. **Data transmission to sink nodes:** The sensor nodes transmit the sensing data to their affiliated sink nodes.



(a) Model Training and Evaluation Phase

(b) Model Deployment and Operation Phase

Fig. 3. Phases of system deployment and operation.

3. **Data validation at sink nodes:** Upon receipt, the sink nodes employ the integrated client model to validate the received sensing data stream.
4. **Sending data to the gateway:** The sink nodes forward the validated sensing data stream, along with the corresponding network traffic data, to the gateway.
5. **Gateway-level validation:** The gateway applies model aggregation prediction algorithms, in conjunction with the client and server models, to conduct a final validation of both the sensing data and the corresponding network traffic data.
6. **Uploading server-validated data:** Following validation, end-users upload the server-validated data to the Internet for additional analysis and processing.

The proposed SC-MLIDS framework is designed to implement two-layer intrusion detection, targeting both the sink node and gateway levels. This approach is particularly applicable considering the inherent vulnerabilities of WSN nodes to network attacks, largely due to their limited computational capacity for deploying robust intrusion-resistant mechanisms. Furthermore, the SC-MLIDS framework demonstrates flexibility in its applicability to small or simple WSNs. In such configurations, the structure is simplified to include a server and a client. Here, the sensor node gathers sensing data and forwards it to the sink node for initial validation. The sink node then transmits this validated data to the gateway, where comprehensive validation of both the sensing data and network traffic data is conducted. The essential models for the aggregation prediction algorithms in this scenario include a client model and a server model. This simplified structure effectively maintains the two-layer intrusion detection characteristic of the SC-MLIDS framework, thereby ensuring robust security even in less complex WSN configurations.

3.2. Aggregation prediction algorithms

The SC-MLIDS framework employs model aggregation methodologies at the gateway level, similar to the bagging approach in EL. This process includes two aggregation prediction algorithms:

1. **Weighted Score Algorithm:** This algorithm combines the prediction results of each model based on their performance metrics and assigned weights. The resulting scores are then converted into binary classification results, ensuring holistic and comprehensive prediction results.
2. **Majority Voting Algorithm:** This algorithm operates on a majority voting principle, where the predictions made by each model are collectively analyzed. The final prediction is then determined based on the results that receive the majority votes among these models.

Due to the flexibility of the proposed two aggregation prediction algorithms, models trained using a variety of classifiers can also be aggregated effectively. This holds regardless of the type and number of classifiers employed. Additionally, the strategy of employing a combination of diverse classifiers, based on specific WSN application scenarios and data characteristics, is also feasible and supported. These algorithms enhance the robustness and accuracy of the SC-MLIDS framework by using the diverse strengths of the individual models involved. The use of both weighted score and majority voting provides a comprehensive approach to model prediction, ensuring more reliable and effective intrusion detection in WSNs.

Considering that both algorithms perform the same task, it is crucial to choose a suitable algorithm. Therefore, it is necessary to examine their performance based on WSN application scenarios and special requirements and select the appropriate algorithm that best suits the current needs. This will ensure that the SC-MLIDS framework achieves optimal intrusion detection in different WSN environments.

3.2.1. Weighted score algorithm

Given the unique characteristics of WSNs, Precision emerges as a particularly crucial metric in assessing the performance of ML models used for such networks. In WSNs, the authenticity and accuracy of data are crucial for subsequent data analysis processes. Consequently, ML models for WSN applications should prioritize minimizing false positives. This focus on Precision reflects the lower relative significance of false negatives in this context.

Precision is defined as the proportion of all samples predicted to be positive that are actually positive. A higher Precision indicates that the model performs greater caution in its predictions, effectively reducing the occurrence of false positives. Additionally, Recall or True Positive Rate (TPR) denotes the proportion of actual positive samples correctly identified as positive by the model. The F1-score addresses potential imbalances within the dataset by harmonizing Precision and Recall. This score is particularly relevant when dealing with unbalanced datasets. Therefore, while Precision remains a critical metric for the performance evaluation of ML models in WSNs, the F1-score also deserves consideration, especially in scenarios involving dataset imbalance.

The weighted score algorithm proposed in this paper innovatively integrates model performance metrics into the bagging methodology of EL, designed specifically for the characteristics of WSNs. As presented in Algorithm 1, this algorithm synthesizes the final prediction results by considering a range of inputs. These inputs include the individual prediction results of each model, the Precision and F1-score of these models, their respective weight shares, and the overall weight assigned to each model.

Algorithm 1: Weighted Score Algorithm

Input : $preds$, $metrics$, $weights$, p_w, f_w
Output: $final\_preds$

$w\_scores = [\ ]$
for m *in* $metrics$ **do**
  $score \leftarrow m[0] \times p\_w + m[1] \times f\_w$
  $w\_scores.append(score)$
end for
$w\_preds = [\ ]$
for $i \leftarrow 0$ **to** *length of* $preds$ *- 1* **do**
  $w\_pred \leftarrow preds[i] \times weights[i] \times w\_scores[i]$
  $w\_preds.append(w\_pred)$
end for
$res \leftarrow$ sum of w_preds / sum of $(weights \times w\_scores)$
$threshold \leftarrow 0.5$
if $res \geq threshold$ **then**
  $final\_pred \leftarrow 1$
else
  $final\_pred \leftarrow 0$
end if
return $final\_preds$

In the proposed weighted score algorithm, the initial step involves running each client model and the server model to generate their respective independent predictions. Following this, the Precision and F1-score obtained during the testing phase of each model are provided. The weights are assigned to each model. These weights are represented as an array, with the condition that their cumulative sum equals 1. This array configuration is crucial as it reflects the relative importance and performance of each model within the aggregation prediction process. For instance, in the SC-MLIDS framework, the server model is typically assigned a greater weight compared to client models. This is due to the server model being trained using network traffic data, which often makes it more critical in the overall predictive accuracy. Finally, the algorithm requires the assignment of weights to the Precision and F1-score. The weight is determined based on the characteristics of the dataset used to train the models. For example, the weights might be assigned as 0.6 and 0.4 for Precision and F1-score, respectively. This weighting approach allows for a balance between Precision and F1-score, ensuring that the aggregation prediction is not only accurate but also sensitive to the dataset's unique properties. After completing the preparation of all required inputs, the weighted score for each model is calculated as a linear combination of its Precision and F1-score.

Let p_i and f_i represent the Precision and F1-score for the i^{th} model, respectively. The weighted score for each model is then given by:

$$score_i = p_i \cdot p_w + f_i \cdot f_w \tag{4}$$

where p_w and f_w are the coefficients associated with Precision and F1-score.

Let $pred_i$ represent the array of binary predictions from the i^{th} model, and w_i be its assigned weight. The final weighted prediction w_pred_i for each model are calculated as:

$$w\_pred_i = pred_i \cdot w_i \cdot score_i \tag{5}$$

Let n be the number of models. The aggregated prediction A_p is then calculated by normalizing the weighted sum of the predictions:

$$A_p = \frac{\sum_{i=1}^{n} w\_pred_i}{\sum_{i=1}^{n} w_i \cdot score_i} \tag{6}$$

Finally, a threshold θ (commonly 0.5 for binary classification) is applied to A_p to obtain the final binary classification P_{weighted}:

$$P_{\text{weighted}} = \begin{cases} 1 & \text{if } A_p \geq \theta \\ 0 & \text{otherwise} \end{cases} \tag{7}$$

In summary, the proposed weighted score algorithm offers a comprehensive approach for scoring model performance and weights in WSNs. This is achieved by considering various factors: the individual prediction results of each model, the performance metrics of the models, the weights assigned to each model, and the assignment of weights among these metrics. A notable aspect of this algorithm is that the final prediction tends to be more closely aligned with models assigned higher weights. Furthermore, if a model with a high weight also gives high-performance metrics, the algorithm's predictions are possible to closely mirror the metrics of that model. However, it is important to acknowledge that due to the aggregate nature of the predictions, there may be cases where the algorithm's performance metrics are slightly lower than those of the best-performing individual model.

The proposed weighted score algorithm is specifically designed for WSNs. By incorporating both Precision and F1-score, along with their respective weights, the algorithm is highly applicable to WSN environments where minimizing false positives is a critical concern. Despite the applicability of this algorithm is not limited to WSNs. With appropriate modifications, such as the integration of additional model metrics or alterations of Precision and F1-score, the algorithm can be adapted to other application scenarios requiring model aggregation prediction.

Therefore, in this paper, the algorithm has been optimized for WSNs to accommodate their specific characteristics and needs. As an indispensable component of the SC-MLIDS framework, it presents a robust and effective method for model aggregation prediction, enhancing the performance of two-layer IDSs in WSNs. This adaptability and precision emphasize the potential of the algorithm as an option in various model aggregation scenarios.

3.2.2. Majority voting algorithm

Majority voting, a frequently employed method in EL, operates on the principle of using predictions from multiple models, with the class receiving the majority of votes being selected as the final predictions. However, considering the specificities of the SC-MLIDS framework, especially within the context of WSNs, a direct application of majority voting is not feasible and requires modification to align with the server–client architecture.

In WSNs applying the SC-MLIDS framework, the diversity of training sets and the limited data collection scope of individual client models at each sink node may not fully cover the range of potential scenarios in data collection. Consequently, an aggregated prediction using the client models from each sink node, combined with majority voting, yields more accurate predictions. In addition, the server model, trained on a comprehensive network traffic dataset, tends to have superior performance in detecting various network traffic. To accommodate these considerations, this paper proposed a modified approach of partial majority voting, as presented in Algorithm 2. In this algorithm, majority voting is primarily based on the prediction results of the client models, but the server model's predictions carry a decisive weight and influence due to its comprehensive training and superior performance.

In the proposed majority voting algorithm, each client and server model independently generates prediction results. Upon inputting the prediction results from each model into the majority voting algorithm, the algorithm proceeds to compute the predictions made by each model. Since this algorithm primarily addresses binary classification

Algorithm 2: Majority Voting Algorithm

```
Input  : preds
Output: final_preds
votes = [ ]
for i ← 0 to length of preds[0] do
    r_sum ← 0
    for j ← 0 to length of preds - 1 do
        r_sum ← r_sum + preds[j][i]
    end for
    threshold ← 2/3 × (length of preds - 1)
    if r_sum ≥ threshold then
        vote ← 1
    else
        vote ← 0
    end if
    votes.append(vote)
end for
final_preds = [ ]
for k ← 0 to length of votes do
    if votes[k] = preds[−1][k] then
        final_preds.append(votes[k])
    else
        final_preds.append(1)
    end if
end for
return final_preds
```

models, the prediction results can be straightforwardly computed. For instance, in the case of three client models, the summation of predictions for a specific sample could yield counts such as 0, 1, 2, or 3. These numbers represent the frequency with which the sample is predicted as the positive class (denoted as 1) across the three models.

To account for potential errors and ensure robustness, the majority voting algorithm adopts a threshold for majority determination. Particularly, a prediction result is considered a majority if it is indicated as a positive class in more than two-thirds of the client models. The majority voting results are then matched with the predictions made by the server model. If there is a consensus between the majority voting result and the server model's prediction, this result is denoted as the final prediction. Otherwise, the algorithm defaults to recording the prediction as a positive class.

For each prediction sample i, the algorithm sums up the predictions from all models except the last one (server model). Let p_{ji} denote the prediction of the j^{th} model for the i^{th} sample. The sum s_i for each sample is given by:

$$s_i = \sum_{j=1}^{n-1} p_{ji} \tag{8}$$

where n is the total number of models, and $n-1$ represents all models except the last one.

The algorithm then applies a threshold θ (generally two-thirds of the number of client models) to this sum to determine the majority vote $vote_i$ for each sample:

$$vote_i = \begin{cases} 1 & \text{if } s_i \geq \theta \\ 0 & \text{otherwise} \end{cases} \tag{9}$$

This reflects the rule that if more than two-thirds of models predict as 1, the majority vote is 1; otherwise, it is 0.

The final prediction P_{voting} for each sample is determined by comparing the majority vote with the prediction of the server model:

$$P_{\text{voting}} = \begin{cases} vote_i & \text{if } vote_i = p_{ni} \\ 1 & \text{otherwise} \end{cases} \tag{10}$$

Here, p_{ni} is the prediction of the server model for the i^{th} sample. If the majority vote matches the server model's prediction, it is taken as the final prediction; otherwise, the final prediction defaults to 1.

Distinct from general majority voting and weighted majority voting approaches, the proposed majority voting algorithm uniquely assigns a decisive voting weight to the model showing superior performance. This customized approach aligns with the server–client structure inherent in the SC-MLIDS framework. It ensures that the predictions made by the client model are co-verified by the server model. The jointly verified results from these two model types are then adopted as the final prediction results. This approach is particularly effective in WSNs, where it is crucial to minimize false positives in data.

The proposed majority voting algorithm, as configured in this paper, is predisposed to yield results leaning toward the positive class. This bias is intentional, deriving from the algorithm's design where discrepancies between the two types of model predictions default to the positive class, thereby reducing the occurrence of false positives. However, this configuration is not rigid; the algorithm's matching conditions and majority voting thresholds can be customized to suit various application scenarios. For instance, adjusting the decisive role's voting weight or altering the majority voting threshold can modify the tolerance for the prediction result.

In conclusion, this paper has optimized the majority voting algorithm to address the specific needs of WSNs, particularly their emphasis on minimizing false positives. As an integral part of the SC-MLIDS framework, this algorithm presents an additional secure and effective method for model aggregation prediction, demonstrating its adaptability and utility in diverse scenarios beyond WSNs.

3.3. Multimodality in the SC-MLIDS framework

Our proposed SC-MLIDS framework integrates ensemble techniques and multimodality to offer a high degree of flexibility and adaptability, as illustrated in Fig. 4.

As a data-driven Artificial Intelligence (AI) intrusion detection framework, SC-MLIDS contains clusters of models based on two data types: application/sensing data and network data. The framework consists of a client model cluster trained on application/sensing data and a server model cluster trained on network data. These model clusters can be trained with either the same or different AI algorithms, allowing for both homogeneous and heterogeneous model structures. In this context, homogeneous models refer to those where all models within a cluster use the same AI algorithm (e.g. RF), while heterogeneous models involve different AI algorithms within the same cluster.

Within the SC-MLIDS architecture, multimodal combinations can be classified into three categories: homogeneous–homogeneous, hetero geneous–heterogeneous, and homogeneous–heterogeneous. These categories represent various combinations of single or multiple AI algorithms used across client and server model clusters. Once the model clusters are constructed, they are fused into a hybrid model, and the final prediction is produced using one of two aggregation prediction algorithms proposed in this paper: the Weighted Score algorithm or the Majority Voting algorithm.

In scenarios where the two proposed algorithms may not fully meet specific application needs, other aggregation prediction algorithms can be selected or developed to generate the final predictions. By leveraging its ensemble and multimodal fusion techniques, the SC-MLIDS framework offers flexible adaptation to WSNs of varying scales and architectures, providing a novel AI-driven solution for intrusion detection in WSNs and opening up new research directions in this field.

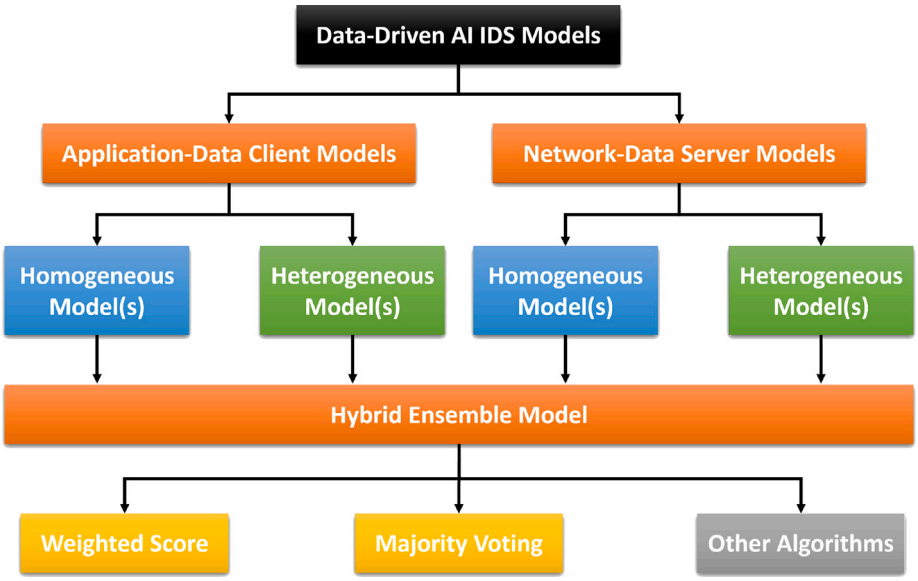


Fig. 4. Multimodality framework.

4. Experimental setup and results

4.1. Integration of datasets

Due to the unique characteristics of the framework proposed in this paper, specific requirements are set for the dataset to be used. The dataset must be generated in a WSN scenario, including sensing and network traffic data, and labeled according to network attacks. However, finding relevant datasets for WSNs that meet these criteria is challenging. Since the WSN is one of the fundamental technologies supporting the IoT, and both involve wireless devices equipped with sensors, selecting a suitable IoT dataset presents an optimal alternative. The TON_IoT [59] dataset contains data from IoT and Industrial IoT sensor devices, as well as operating system and network traffic data, making it suitable for the validation of ML network security solutions for IoT, such as IDSs.

In this paper, the IoT device sensing data and network traffic data from the processed dataset are employed. The weather dataset within the IoT device sensing data, which closely aligns with the WSN scenario, is the only dataset used. Conversely, the network traffic dataset is used in its entirety, as it is not categorized based on specific IoT devices.

The IoT weather dataset collects weather sensing data including temperature, pressure, and humidity between March 31 and April 27, 2019, and is labeled as normal and seven attacks (Backdoor, Password, DDoS, Injection, Ransomware, XSS, and Scanning). The dataset has a total of 650,242 samples, 86%, amounting to 559,718 samples, representing normal data, while the remaining 14% represents data affected by each of the seven types of attacks.

The network traffic dataset is complex, containing network traffic from various devices. It consists of 21,978,631 samples and 46 features across 23 dataset files. The dataset is labeled as normal and nine types of attacks (Scanning, DDoS, DoS, XSS, Password, Backdoor, Injection, Ransomware, and MITM), with only about 3.6% of the data (corresponding to 788,599 samples) being normal, and the remaining 21,190,031 samples representing various types of attack data.

By merging the IoT weather dataset and the network traffic dataset based on common features, and subsequently performing cleaning, encoding, scaling, and shuffling operations on the merged dataset, we obtained the final dataset. The generated dataset is presented in Table 3. It combines sensing data from the IoT weather dataset with network connection activity and statistical activity data from the network traffic dataset.

In this paper, the proposed framework contains a server and multiple clients. A dataset split that reflects this structure is required, beyond merely dividing into training and testing sets. To demonstrate and test the proposed framework, we use three clients as an example. The process of dataset splitting is illustrated in Fig. 5. Initially, the merged dataset is divided into a training set and a testing set, with a ratio of 80% and 20%, respectively.

Considering that the merged dataset includes both sensing data and network traffic data, it is essential to split the dataset in alignment with the distinct functions of the server and the clients. Specifically, the server requires only network traffic data, while the clients need only sensing data. Therefore, the training set is further divided based on data type, ensuring the label column is retained in both subsets. The obtained datasets are classified as the sensor dataset and the network dataset. The sensor dataset is evenly split among the three clients for this experiment. Through these steps, four distinct datasets are generated: the network dataset for the server, and three sensor datasets for the clients.

4.2. Implementation details

To demonstrate the practical viability of the proposed SC-MLIDS framework, we have developed a Python-based simulation program to simulate the communication between sink nodes and the gateway in WSNs. In our simulation, the SC-MLIDS framework's functionality is showcased during both the deployment and operation phases of WSNs. The program executes model training, testing, and transmission during the model training and evaluation phase. It manages sensing data collection, validation, transmission, file processing, and aggregated validation in the model deployment and operation phase. This program is partitioned into four modules: Utilities, Server, Client, and Aggregation Prediction. Each module's code and functionality are elaborated as follows:

- **Utility Module:** This module is an auxiliary tool within the simulation program. It contains a variety of functions, including importing and splitting datasets, outputting distributions of dataset labels, generating model performance metrics, and creating encryption keys.
- **Server Module:** Designed to simulate the gateway's role in a WSN, this module handles communication with clients. It manages the reception of client messages, including metadata, ML

Table 3
Generated dataset.

Index	0	1	2	3	4	...	203 440
temperature	0.9486	0.5170	0.9596	0.5819	0.5132	...	0.9509
pressure	0.6904	0.5001	0.4670	0.6743	0.7438	...	0.4022
humidity	0.2075	0.4147	0.4953	0.5789	0.7367	...	0.6414
src_port	0.8152	0.6586	0.8236	0.1202	0.8618	...	0.7081
dst_port	0.0068	0.0008	0.1603	0.6426	0.1233	...	0.0012
Proto	1	2	1	1	1	...	1
Service	9	3	0	0	0	...	0
Duration	0	0	0	0	0	...	0
src_bytes	0	0	0	0	0	...	0
dst_bytes	0	0	0	0	0	...	0
conn_state	4	10	0	0	0	...	10
src_pkts	0	0	0	0	0	...	0
src_ip_bytes	0	0	0	0	0	...	0
dst_pkts	0.0001	0	0	0	0	...	0
dst_ip_bytes	0.0001	0	0	0	0	...	0
target	1	0	0	1	1	...	1

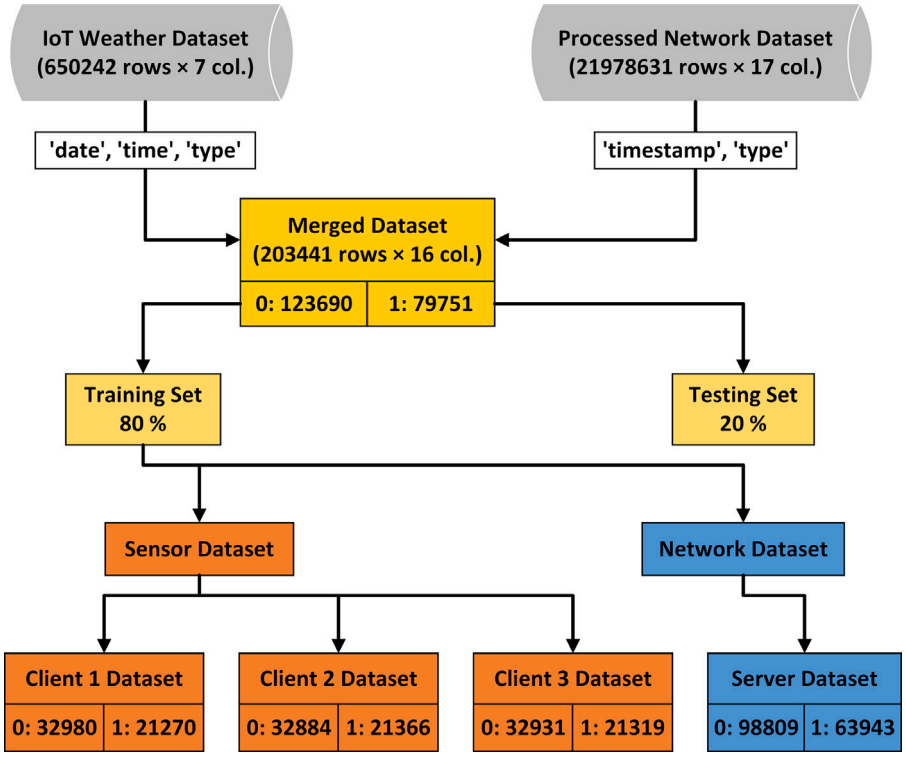


Fig. 5. Dataset processing, integration and splitting.

model files, and subsequent sensing and network traffic data. Additional functionalities include the decryption and decompression of model files and the training of the server model.

- **Client Module:** Simulating the sink node's function in a WSN, this module facilitates communication with the server. The client's primary tasks include training the client model, compressing and encrypting the model, and then transmitting it to the server. The module also handles the transmission of sensing data to the server. In scenarios involving multiple clients, the client module can be duplicated and modified with distinct client IDs to simulate various client entities.
- **Aggregation Prediction Module:** This module implements the two aggregation prediction algorithms specifically designed for the SC-MLIDS framework. Additionally, it includes a helper method for integrating and generating the necessary inputs for these algorithms and for displaying the performance metrics of the models.

In our simulation program, Random Forest (RF) classifiers are uniformly used at both the client and server levels. The choice of RF is driven by their typically high accuracy, robust performance, and efficiency, particularly with unbalanced datasets and a multitude of predictive variables [60–62]. Furthermore, their widespread support across numerous open-source libraries facilitates ease of use and avoids the need for extensive parameter tuning. Given these advantages, the RF classifier is consistently used throughout our simulation.

However, it is critical to note that the SC-MLIDS framework and its integrated model aggregation prediction algorithms are not restricted to this single classifier type. The framework is flexible, supporting the integration of various other classifiers depending on specific application scenarios and data characteristics. Moreover, due to the adaptability of the proposed aggregation prediction algorithms, models developed using diverse classifiers can be effectively aggregated, independent of their type and quantity.

4.3. Experimental setup

The experimental phase was executed on a laptop equipped with an Intel Core i9-13900H CPU and an NVIDIA GeForce RTX 4060 Laptop GPU. The laptop also features 32 GB of DDR5 memory and a 1 TB SSD, running Microsoft Windows 11 Home 64-bit.

The core aim of our simulation program is to demonstrate client–server interaction when the SC-MLIDS framework is applied to a WSN environment. We use the collected sensing data to train the client model at the client. At the server, we use network traffic data for training the server model and implementing the model aggregation. To maintain data integrity and security throughout this process, the client models are compressed and encrypted before their transmission to the server. To simulate local communication on the laptop, the host and port numbers for both the client and server were uniformly set to '127.0.0.1' and '8080', respectively.

This experimental setup included one server and three clients. The processed dataset was divided such that 80% was allocated as the training set and the remaining 20% as the testing set. Each client was assigned one-third of the training set, while the server used the entirety of this set.

In terms of evaluation, each model's performance was assessed using metrics such as Accuracy, Precision, Recall, and F1-score. Additionally, we implemented the two proposed model aggregation prediction algorithms to merge the prediction results of these models. This approach aimed to enhance the overall detection efficiency and accuracy, yielding comprehensive prediction results that are not solely dependent on a single model's performance.

The proposed weighted score algorithm is based on model performance and weights, the weights assigned to the three client models were set uniformly at 0.2 each, while the server model was assigned a higher weight of 0.4, reflecting its greater importance and performance. In terms of performance metrics, particular attention was given to the Precision and F1-score. The weight assigned to Precision was set at 0.6, while the F1-score was weighted at 0.4. This specific weighting strategy was adopted because we emphasized minimizing false positives in the model predictions. Additionally, it reflects a balanced trade-off in the model's predictive accuracy, considering both the precision and the holistic performance as represented by the F1-score.

To support reproducibility, we provide a simplified version of our early experiments in our GitHub repository [63]. While the experiments in this paper were conducted using an extended version of this code, the repository will be updated to reflect subsequent improvements and additional experiments.

4.4. Results

The proposed SC-MLIDS framework is characterized by two key software components: server–client communication and model aggregation prediction algorithms. To facilitate a comprehensive evaluation of the SC-MLIDS framework, our experimental design was separated into two distinct parts, corresponding to the two software components.

4.4.1. Server–client communication

In our experimental evaluation of the SC-MLIDS framework, we simulated and analyzed the communication process between the server and three clients using a developed simulation program.

Table 4 systematically presents and summarizes this process. Each client model is trained using sensing data, then encrypted and compressed before being sent to the server. The server, upon receiving these model files, proceeds to decompress and decrypt them. Subsequently, the server model is trained using network traffic data. To ensure uniformity across the experiment, all models uniformly employed the RF classifier.

For this local transmission simulation, clients were configured with loopback addresses but with distinct ports. Each client was assigned a training set of roughly equivalent size, though variations in label distribution were observed. The model training time for all three clients averaged just over 8 s. In contrast, the server, processing three times the amount of data, completed its model training in merely 6.83 s. The server model file, not being transmitted, was exempt from encryption and compression procedures. The generated model files across the clients were approximately 106 MB in size. Encryption of these files was fast, ranging from 0.5654 to 0.7266 s, but resulting in a file size increase to about 142 MB. After compression, the model files were approximately 107 MB, slightly larger than the original size, with compression times varying from 3.8393 to 5.2522 s for the clients.

The transfer of the encrypted and compressed model files to the server was executed with remarkable speed, showcasing the efficiency of the communication process. The consistency in the size of the files received by the server compared to those sent by the clients confirmed the integrity of the data during transmission. However, notable variations were observed in the reception times. Specifically, Client 1 experienced a longer reception time of approximately 80 s, while Client 2 completed the transfer the quickest, in about 65 s, and Client 3 fell in between with a duration of 76 s. The average transmission rate for all clients exceeded 1 MB/s, with Client 2 achieving the highest rate at 1.66 MB/s. During the final phases of decompression and decryption, each client completed the task in approximately 1 s, further demonstrating the framework's effectiveness in secure model transmission.

For a preliminary evaluation of the energy consumption of the SC-MLIDS framework, we monitored and collected the energy consumption data of RAM, CPU, and GPU during the experiments. The results are summarized in Table 5. In the three-client experimental scenario, the total runtime of the simulation program is 266 s. The RAM and CPU usage range between 1.2% and 4.4%, indicating low overall power consumption. The RAM consumption is consistent across the three clients at 0.000037 kWh, while it is slightly higher for the server at 0.000875 kWh. CPU energy consumption follows a similar trend, with clients consuming 0.000133 kWh and the server consuming 0.003129 kWh. Additionally, the GPU power consumption is minimal, as the program primarily runs on the CPU. The total power consumption of the simulated program is 0.004892 kWh, leading to an estimated carbon dioxide emission of 0.000192 kg. These simulation results demonstrate the high energy efficiency of SC-MLIDS and provide a reference for resource-constrained WSN scenarios in terms of application power consumption.

We also conducted experiments using different combinations of classifiers, partial results are shown in Table 6. This included a selection of popular classifiers such as RF, Logistic Regression (LR), Gradient Boosting (GB), and Support Vector Machine (SVM).

Despite using the same training sets, there was a marked difference in the file sizes of the models generated by the different classifiers. Client 1, using the RF classifier, produced a model file size similar to the previous round, approximately 108 MB. Client 2, using LR, generated a significantly smaller model file of only 2 KB. Client 3, using GB, produced a model file of 188 KB. The server model trained using SVM, resulted in a file size of only 1.39 MB. These results highlight that the choice of classifier not only impacts the size of the resulting model file but also influences the efficiency of transmission, thereby affecting the overall performance.

4.4.2. Aggregation prediction algorithms

To thoroughly evaluate the model aggregation prediction algorithms within the proposed SC-MLIDS framework, we initially assessed the independent performance of each model. This assessment involved deriving metrics such as Accuracy, Precision, Recall, and F1-score.

As shown in Fig. 6(a), the performance metrics of the RF-based models reveal their high effectiveness. The three client models consistently show performance metrics slightly above 0.93 across Accuracy, Precision, Recall, and F1-score. This consistency indicates their robust capability in accurate prediction. The close alignment of all four metrics

Table 4
Model training and transmission results (random forests).

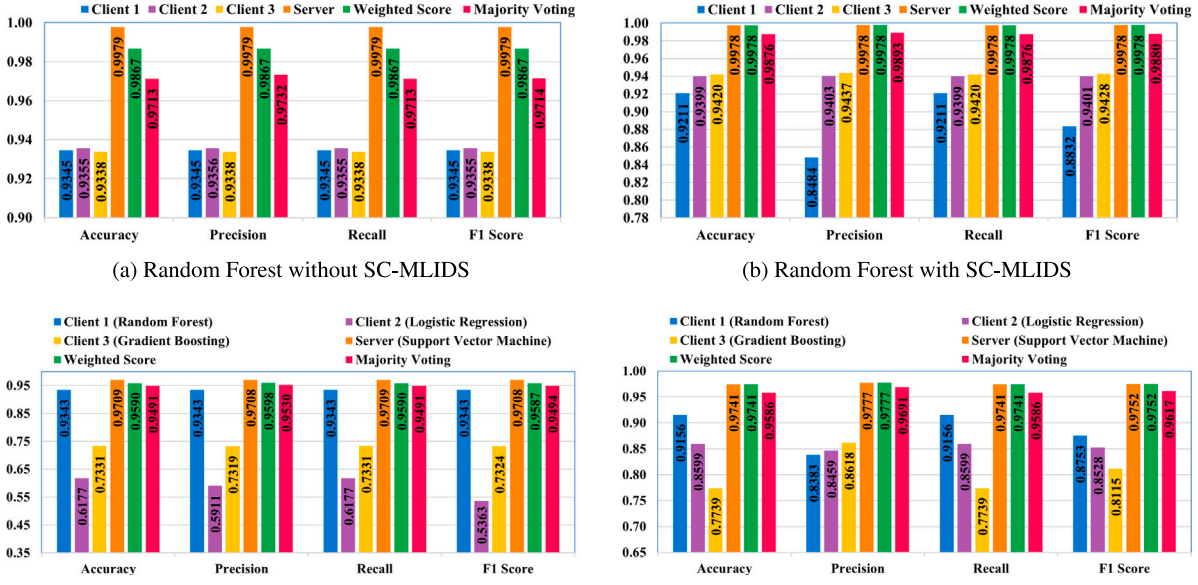
Metric	Server	Client 1	Client 2	Client 3
Label distribution	98809, 63943	32980, 21270	32884, 21366	32931, 21319
Model training time (s)	6.8305	8.9069	8.3103	8.1050
Model file size (MB)	18.7705	106.4463	106.6699	106.2451
Encrypted size (MB)	N/A	141.9277	142.2266	141.6600
Encryption time (s)	N/A	0.7266	0.5947	0.5654
Compressed size (MB)	N/A	107.5145	107.7409	107.3119
Compression time (s)	N/A	5.2522	3.8984	3.8393
Sending time (s)	N/A	0.0100	0.0080	0.0080
Received file size (MB)	N/A	107.5145	107.7409	107.3119
Receiving time (s)	N/A	80.2155	64.8709	76.1133
Average rate (MB/s)	N/A	1.3403	1.6609	1.4099
Decompression & Decryption time (s)	N/A	1.1003	0.8955	1.1386

Table 5
Energy consumption analysis.

Metric	Client 1	Client 2	Client 3	Server	Total
CPU usage	1.80%	4.40%	3.20%	2.40%	–
Memory usage	2.30%	2.10%	2.10%	1.20%	–
RAM energy (kWh)	0.000037	0.000038	0.000037	0.000875	0.000987
CPU energy (kWh)	0.000133	0.000136	0.000133	0.003129	0.003531
GPU energy (kWh)	0.000012	0.000015	0.000012	0.000334	0.000373
RAM power (W)	11.8989	11.8989	11.8989	11.8989	–
CPU power (W)	42.5	42.5	42.5	42.5	–
GPU power (W)	3.7535	4.8117	3.7656	4.4316	–
CO_2 emission (kg)	0.000007	0.000007	0.000007	0.000171	0.000192
Total energy (kWh)	0.000183	0.000190	0.000182	0.004337	0.004892
Execution time (s)	0.030843	0.039218	0.025392	265.868227	265.9637

Table 6
Model training results using different classifiers.

Metric	Server	Client 1	Client 2	Client 3
Classifier	Support vector machine	Random forest	Logistic regression	Gradient boosting
Label distribution	98809, 63943	32980, 21270	32884, 21366	32931, 21319
Training time (s)	90.806	8.622	0.206	4.672
File size (MB)	1.3936	108.0674	0.0020	0.1836



(a) Random Forest without SC-MLIDS

(b) Random Forest with SC-MLIDS

(c) Multiple Classifiers without SC-MLIDS

(d) Multiple Classifiers with SC-MLIDS

Fig. 6. Comparison of random forest and multiple classifiers with and without SC-MLIDS.

for each model suggests a well-balanced proficiency in identifying both positive and negative classes. The server model demonstrates nearly perfect scores in all metrics, exceeding 0.99, reflecting its superior predictive ability.

However, the reliance on a single model type is insufficient in the SC-MLIDS framework, which integrates server and client models, each validating different data types. Hence, we implemented two model aggregation prediction algorithms to yield balanced and comprehensive

prediction results. Fig. 6(a) also presents the results of the weighted score algorithm, a scoring algorithm based on model performance and weights. The weights for the three client models and the server model were set at 0.2, 0.2, 0.2, and 0.4, respectively, reflecting the server model's higher performance and importance. The Precision weight was set at 0.6 and the F1-score weight was set at 0.4. This weighting strategy was adopted because we emphasized minimizing false positives and achieving balanced predictions. The weighted score algorithm combines the models to yield predictions that are tested on the testing set with all four metrics at approximately 0.9867. Similarly, the majority voting algorithm performs robustly, exceeding 0.97 in all metrics, with a slightly higher Precision. While both algorithms slightly lag behind the best-performing server model (by about 1% and 2%), they offer a more comprehensive intrusion detection capability, reflecting the integrated predictive power that individual models lack.

In the next phase of our experiment, we continued to evaluate algorithm performance by simulating the operation phase of the SC-MLIDS framework. Specifically, we simulated a sink node collecting data from its managed sensor nodes, validating it using the client model at the sink node, and then forwarding it to the gateway for aggregation prediction. For this experiment, we used a balanced subset of the testing set, giving an example of Client 1, randomly selecting 10,000 samples each from positive and negative classes. Client 1 predicted 9739 samples as positive and discarded them. Of the 10,261 samples predicted as negative and sent to the gateway, 9451 were true negatives, and 810 were false negatives.

Fig. 6(b) details the results of the aggregation prediction performed at the gateway. As expected, when data is validated by Client 1 with further filtering and validation, we observe normal Accuracy and Recall metrics of 0.92, with lower Precision and F1-score. The other two client models gave slightly higher metrics, around 0.94. The performance of the aggregation prediction algorithms gave significant improvement when the SC-MLIDS framework was applied. Compared to previous testing, both algorithms showed improvements in all four metrics, by 1.12% and 1.68%, respectively. The weighted score algorithm's performance was identical to the best-performing server model, with all four metrics exceeding 0.99.

In our further experiment with the proposed SC-MLIDS framework, we explored using various classifiers for generating models on the example of three clients, as shown in Fig. 6(c). The classifiers deployed were RF for Client 1, LR for Client 2, and GB for Client 3, while the server model was trained using the SVM algorithm. Client 1 maintained the same performance metrics as the previous experiment due to the continued use of the RF classifier. The other two clients, however, showed poorer performance, with all metrics ranging between 0.53 and 0.73. In contrast, the server model demonstrated the highest performance, achieving an accuracy and other metrics as high as 0.97. When these models were merged using the two aggregation prediction algorithms, the synthesized prediction results for both algorithms were approximately 0.96 and 0.94 across all four metrics, respectively. Although these results were slightly lower than the best-performing server model, the aggregation achieved high accuracy in predictions, effectively harmonizing the individual models with varied performances.

Fig. 6(d) presents the results of the simulation runs for the SC-MLIDS framework using Client 1 as an example. After filtering the sensing data, the metrics showed varying degrees of improvement when predicted by each model. The most notable improvement was observed in Client 2, exceeding 20%. This improvement also positively impacted the performance of the model aggregation prediction algorithms, with both algorithms improving each metric by more than 1% compared to individual tests.

These experiments collectively demonstrate the performance of the SC-MLIDS framework and its aggregation prediction algorithms from multiple angles. When operating a WSN under the SC-MLIDS framework, data forwarded to the gateway experiences a two-layer validation process: first by the client model at the sink node and then by the synthesized verification at the gateway. This implementation of the aggregation prediction algorithms results in significantly enhanced accuracy of intrusion detection. Although the algorithm's performance metrics did not achieve a perfect model level, the comprehensive nature of the prediction approach employed ensures more reliable results.

We have examined several state-of-the-art studies, and Table 7 compares the performance of the ML IDS schemes proposed in these studies with the performance of our own. All of these studies utilize the TON_IoT dataset, with some relying on telemetry data, others on network data, and some, like our study, using a combination of both telemetry and network data. The performance metrics of these studies, along with the metrics of the two aggregation algorithms from our study, are presented in the table.

Among these studies, the performance of [68] is the most comparable to ours, with slightly lower Accuracy and Recall but marginally higher Precision and F1-score. The performance of [66] also stands out, achieving 0.9949 across all four metrics, which is lower than the performance of our Weighted Score algorithm (0.9978) but slightly higher than our Majority Voting algorithm (0.9876). While our Weighted Score algorithm outperforms all other studies across all four metrics, the Majority Voting algorithm still surpasses many of the studies.

To verify the generalizability of the SC-MLIDS framework, we conducted experiments on four widely used WSN datasets: WSN-DS [72], SensorNetGuard [73], LWSNDR [74], and WSNBFSF [75]. The results are summarized in Table 8. Since these datasets contain either network traffic data or sensor data, they do not fully cover all functional modules of SC-MLIDS. Specifically, WSN-DS, SensorNetGuard, and WSNBFSF focus on WSN network intrusion detection and contain network traffic data. We evaluated these three datasets on the server-side model, achieving performance exceeding 0.98 across all four key evaluation metrics, with some metrics reaching 1.00. LWSNDR is a sensor dataset containing temperature and humidity data collected by multiple sensors. We selected three sensor datasets as clients and applied our aggregation prediction algorithms for performance evaluation. The results showed 1.00 performance across all four evaluation metrics, conforming the framework's effectiveness in processing sensor data. These findings demonstrate that our proposed framework (SC-MLIDS) generalizes well across different WSN datasets, reinforcing its applicability to diverse real-world scenarios.

In conclusion, despite the strong performance of several other approaches, our SC-MLIDS framework, with its novel model fusion approach, achieves superior results in terms of Accuracy, Precision, Recall, and F1-score. The Weighted Score algorithm, in particular, offers a significant advantage and provides a great advantage in the field of intrusion detection.

5. Discussion

In our study, we conducted experiments to validate the efficacy of the proposed SC-MLIDS framework, focusing on its server–client communication simulation and aggregation prediction algorithms. The simulation program of the SC-MLIDS framework consisted of two main phases: model training and evaluation phase, model deployment and operation phase.

During the model training and evaluation phase, the program showcased processes such as client model training, client–server communication, server model training, and model aggregation prediction. These experiments provided insights into resource consumption, highlighting potential additional consumption in model training and transmission. We noted that for simple sensing tasks, employing simpler algorithms like DT or LR could lead to significant resource savings.

In the model deployment and operation phase, the client model was used for initial data filtering, and discarding malicious data to reduce resource consumption. Model aggregation at the gateway merged client and server models, with the gateway managing the task effectively.

Table 7
Comparison with state-of-the-art techniques.

Paper	Dataset	Proposed framework	Models	Accuracy	Precision	Recall	F1-score
[64]	Telemetry	Ensemble model	XGBoost	0.9878	0.9788	0.9896	0.9840
[65]	Telemetry	ML IDS	Stacking & Voting	0.9864	0.9866	0.9860	0.9861
[66]	Network	ML-based IoT IDS	Stacking	0.9949	0.9949	0.9949	0.9949
[67]	Network	SATIDS	LSTM	0.9656	0.973	0.9740	0.9735
[68]	Network	BiGRU-CNN-based IDS	CNN + GRU	0.9971	0.9989	0.9905	0.9985
[69]	Both	IoT IDS	DNN	0.9298	0.9408	0.9298	0.9308
[70]	Network	Optimized DL IDS	DL	0.9926	0.9918	0.9922	0.9920
[71]	Telemetry	Optimized LSTM	LSTM	0.6800	0.7300	0.6800	0.6300
Our Work	Both	SC-MLIDS	Hybrid (Weighted Score)	**0.9978**	**0.9978**	**0.9978**	**0.9978**
			Hybrid (Majority Voting)	**0.9876**	**0.9893**	**0.9876**	**0.9880**

Table 8
Performance assessment on additional WSN datasets.

Dataset	Features	Samples	Network data	Sensor data	Accuracy	Precision	Recall	F1-score
TON_IoT	16	203,440	✓	✓	0.9978	0.9978	0.9978	0.9978
WSN-DS	19	374,661	✓	✗	0.9972	0.9870	0.9832	0.9851
SensorNetGuard	21	10,000	✓	✗	0.9990	1.0000	0.9798	0.9898
LWSNDR	5	14,421	✗	✓	1.0000	1.0000	1.0000	1.0000
WSNBFSF	18	312,106	✓	✗	0.9999	1.0000	0.9996	0.9998

Our experiments indicated that the SC-MLIDS framework stayed within WSN resource limitations, affirming its alignment with the need for lightweight WSN solutions.

The gateway's role is critical in SC-MLIDS, aggregating predictions from client and server models to generate final detection results. This aggregation is facilitated by two algorithms: Weighted Score and Majority Voting. The weighted score algorithm combines Precision and F1-score with weights based on model importance or task complexity, addressing false positives in WSNs. The majority voting relies on collective client model voting, matched with server model results for reliability.

Since both algorithms perform model aggregation, one of them should be considered or dynamically selected depending on the WSN application scenario and the performance. Testing revealed that the server model, benefiting from a complete training set, outperformed client models slightly. However, both algorithms still achieved excellent performance metrics, meeting or exceeding 0.99. Their adaptability makes them effective even with underperforming models or complex tasks.

The SC-MLIDS framework aligns with the typical three-layer architecture of WSNs, providing intrusion detection at sink node and gateway levels. This design flexibility allows adaptation to varying WSN architectures, offering an innovative security solution. Moreover, the framework efficiently utilizes computational and network resources, meeting the WSN's need for lightweight solutions.

The proposed SC-MLIDS framework has several limitations. Firstly, the TON_IoT dataset used may not fully reflect real-world WSN scenarios due to the absence of suitable WSN datasets containing both sensing and network traffic data. Our method of merging these data types based on matching timestamps and labels could introduce errors and biases. Secondly, our simulations might not accurately replicate real-world WSN deployments, potentially missing complexities that could arise in practical applications. Lastly, the current model aggregation prediction algorithms do not adequately address the impact of low-precision models, which could reduce intrusion detection confidence.

6. Conclusion and future work

The SC-MLIDS proposed in this paper presents an advanced hybrid ML framework for intrusion detection in WSNs that leverages multi-source data fusion. By integrating data from multiple sensor nodes and network layers, SC-MLIDS minimizes data transmission while ensuring high-precision detection. Designed with WSN architectural specifics in mind, SC-MLIDS employs a two-layer detection approach, utilizing simplified client models at sink nodes and a more complex server model at the gateway. This approach balances low resource consumption with enhanced detection accuracy, effectively addressing the challenges of multi-source data integration. We validated the framework through comprehensive experiments, evaluating the performance of individual models and the fusion capabilities of our two proposed aggregation prediction algorithms.

The proposed aggregation prediction algorithms, Weighted Score and Majority Voting, are tailored to enhance prediction reliability through effective multi-source data fusion. These algorithms address WSN-specific characteristics, such as data features and transmission constraints, achieving efficient intrusion detection with a low false positive rate. Through our experiments, the proposed two model fusion methods demonstrated superior performance across Accuracy, Precision, Recall, and F1-score. Specifically, the Weighted Score algorithm achieved 0.9978 in all four metrics, while the Majority Voting algorithm consistently surpassed 0.9876. When compared to models from current state-of-the-art research, the Weighted Score algorithm outperforms all others, and the Majority Voting algorithm also surpasses many existing approaches. Experimental results highlight the algorithms' proficiency in integrating diverse model predictions, ensuring comprehensive detection across varied performance levels and minimizing reliance on any single model. This fusion approach significantly contributes to the robustness of the SC-MLIDS framework.

The SC-MLIDS framework's design and implementation highlight its effectiveness in multi-source and multi-process data fusion, offering adaptability for diverse attack types and flexibility in ML algorithm selection. This makes SC-MLIDS a significant advancement in WSN security, providing a robust and scalable solution for intrusion detection. The framework not only demonstrates high efficiency and accuracy but also leads the way for future research into integrating advanced ML fusion techniques in WSN security. The results underscore SC-MLIDS's potential to enhance intrusion detection methodologies and inspire further exploration in the field.

Future work should focus on dataset collection, algorithm selection, model selection algorithm for ensemble, and deployment strategy. Deploying real WSNs to collect actual sensing and network traffic data will enhance the framework's accuracy and relevance. Conducting various attacks on these networks will provide a broader range of scenarios, improving model robustness. Experimenting with different ML algorithms is crucial, with simpler algorithms for client models and more sophisticated ones for server models. Enhancing model aggregation prediction algorithms with dynamic weight assignment based on model performance or replacing underperforming models will improve system

performance. Additionally, we plan to develop a WSN simulator that supports creating local models for embedded devices and ensemble models for gateway servers. This simulator will provide practical data during the model validation phase, serving as a reference for real applications. Following simulation and hardware experiments, we intend to deploy the framework in a real-world WSN to evaluate its performance in real scenarios. These improvements would enable the framework to better adapt to increasingly complex network attack threats, ensuring more robust and responsive intrusion detection.

CRediT authorship contribution statement

Hongwei Zhang: Writing – original draft, Visualization, Validation, Methodology, Software, Investigation, Formal analysis, Data curation, Resources, Conceptualization. **Darshana Upadhyay:** Writing – review & editing, Visualization, Validation. **Marzia Zaman:** Writing – review & editing, Supervision, Project administration, Methodology, Investigation, Conceptualization. **Achin Jain:** Writing – review & editing. **Srinivas Sampalli:** Writing – review & editing, Supervision, Project administration, Methodology, Investigation.

Declaration of competing interest

The authors declare the following financial interests/personal relationships which may be considered as potential competing interests: Darshana Upadhyay reports was provided by Natural Sciences and Engineering Research Council of Canada. If there are other authors, they declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

The authors gratefully acknowledge the support in part by the Natural Sciences and Engineering Research Council (NSERC) and industry partners Norleaf Networks and Cistel Technology Inc., through a Collaborative Research Grant.

Data availability

Data will be made available on request.

References

[1] A. Flammini, P. Ferrari, D. Marioli, E. Sisinni, A. Taroni, Wired and wireless sensor networks for industrial applications, Microelectron. J. 40 (9) (2009) 1322–1336.
[2] M.N. Mowla, N. Mowla, A.S. Shah, K. Rabie, T. Shongwe, Internet of things and wireless sensor networks for smart agriculture applications-a survey, IEEE Access (2023).
[3] O. Can, O.K. Sahingoz, A survey of intrusion detection systems in wireless sensor networks, in: 2015 6th International Conference on Modeling, Simulation, and Applied Optimization, ICMSAO, IEEE, 2015, pp. 1–6.
[4] P. Mishra, V. Varadharajan, U. Tupakula, E.S. Pilli, A detailed investigation and analysis of using machine learning techniques for intrusion detection, IEEE Commun. Surv. Tutor. 21 (1) (2018) 686–728.
[5] K. Medhat, R.A. Ramadan, I. Talkhan, Distributed intrusion detection system for wireless sensor networks, in: 2015 9th International Conference on Next Generation Mobile Applications, Services and Technologies, IEEE, 2015, pp. 234–239.
[6] Y. Maleh, A. Ezzati, Y. Qasmaoui, M. Mbida, A global hybrid intrusion detection system for wireless sensor networks, Procedia Comput. Sci. 52 (2015) 1047–1052.
[7] W. Zhang, D. Han, K.-C. Li, F.I. Massetto, Wireless sensor network intrusion detection system based on MK-ELM, Soft Comput. 24 (2020) 12361–12374.
[8] N.M. Alruhaily, D.M. Ibrahim, A multi-layer machine learning-based intrusion detection system for wireless sensor networks, Int. J. Adv. Comput. Sci. Appl. 12 (4) (2021).
[9] H. Zhang, M. Zaman, A. Jain, S. Sampalli, A hybrid machine learning intrusion detection system for wireless sensor networks, in: 2024 International Wireless Communications and Mobile Computing, IWCMC, IEEE, 2024, pp. 830–835.
[10] F. Gaojuan, W. Ruchuan, H. Haiping, S. Lijuan, X. Fu, Performance analysis for intrusion target detection in wireless sensor networks, Chin. J. Electron. 20 (4) (2011) 725–729.
[11] W. Wang, H. Huang, Q. Li, F. He, C. Sha, Generalized intrusion detection mechanism for empowered intruders in wireless sensor networks, IEEE Access 8 (2020) 25170–25183.
[12] D.-f. Ye, L.-l. Min, W. Wang, Design and implementation of wireless sensor network gateway based on environmental monitoring, in: 2009 International Conference on Environmental Science and Information Application Technology, Vol. 2, IEEE, 2009, pp. 289–292.
[13] M. Kocakulak, I. Butun, An overview of Wireless Sensor Networks towards internet of things, in: 2017 IEEE 7th Annual Computing and Communication Workshop and Conference, CCWC, Ieee, 2017, pp. 1–6.
[14] J. Yick, B. Mukherjee, D. Ghosal, Wireless sensor network survey, Comput. Netw. 52 (12) (2008) 2292–2330.
[15] K. Bagadi, R. Cv, K. Sathish, An overview of localization techniques in underwater wireless sensor networks, in: 2022 Third International Conference on Intelligent Computing Instrumentation and Control Technologies, ICICICT, IEEE, 2022, pp. 1687–1692.
[16] H. Guo, K.-S. Low, H.-A. Nguyen, Optimizing the localization of a wireless sensor network in real time based on a low-cost microcontroller, IEEE Trans. Ind. Electron. 58 (3) (2009) 741–749.
[17] A. Khalifeh, F. Mazunga, A. Nechibvute, B.M. Nyambo, Microcontroller unit-based wireless sensor network nodes: A review, Sensors 22 (22) (2022) 8937.
[18] Y. Gu, F. Ren, Y. Ji, J. Li, The evolution of sink mobility management in wireless sensor networks: A survey, IEEE Commun. Surv. Tutor. 18 (1) (2015) 507–524.
[19] I.F. Akyildiz, W. Su, Y. Sankarasubramaniam, E. Cayirci, A survey on sensor networks, IEEE Commun. Mag. 40 (8) (2002) 102–114.
[20] A. Ali, Y. Ming, S. Chakraborty, S. Iram, A comprehensive survey on real-time applications of WSN, Futur. Internet 9 (4) (2017) 77.
[21] C. Buratti, A. Conti, D. Dardari, R. Verdone, An overview on wireless sensor networks technology and evolution, Sensors 9 (9) (2009) 6869–6896.
[22] L.d.T. Steenkamp, S. Kaplan, R.H. Wilkinson, Wireless sensor network gateway, in: AFRICON 2009, IEEE, 2009, pp. 1–6.
[23] J. Kim, D. Choi, Esgate: Secure embedded gateway system for a wireless sensor network, in: 2008 IEEE International Symposium on Consumer Electronics, IEEE, 2008, pp. 1–4.
[24] M.A. Hussain, K. kyung Sup, et al., WSN research activities for military application, in: 2009 11th International Conference on Advanced Communication Technology, Vol. 1, IEEE, 2009, pp. 271–274.
[25] P.-C. Hii, W.-Y. Chung, A comprehensive ubiquitous healthcare solution on an Android™ mobile device, Sensors 11 (7) (2011) 6799–6815.
[26] D.D.K. Rathinam, D. Surendran, A. Shilpa, A.S. Grace, J. Sherin, Modern agriculture using wireless sensor network (WSN), in: 2019 5th International Conference on Advanced Computing & Communication Systems, ICACCS, IEEE, 2019, pp. 515–519.
[27] R.S. Ransing, M. Rajput, Smart home for elderly care, based on wireless sensor network, in: 2015 International Conference on Nascent Technologies in the Engineering Field, ICNTE, IEEE, 2015, pp. 1–5.
[28] S. Mansour, N. Nasser, L. Karim, A. Ali, Wireless sensor network-based air quality monitoring system, in: 2014 International Conference on Computing, Networking and Communications, ICNC, IEEE, 2014, pp. 545–550.
[29] J. Zhang, W. Li, N. Han, J. Kan, Forest fire detection system based on a ZigBee wireless sensor network, Front. For. China 3 (2008) 369–374.
[30] J. Ben-Othman, B. Yahya, Energy efficient and QoS based routing protocol for wireless sensor networks, J. Parallel Distrib. Comput. 70 (8) (2010) 849–857.
[31] M.A.M. Vieira, C.N. Coelho, D.j. da Silva, J.M. da Mata, Survey on wireless sensor network devices, in: EFTA 2003. 2003 IEEE Conference on Emerging Technologies and Factory Automation. Proceedings (Cat. No. 03TH8696), Vol. 1, IEEE, 2003, pp. 537–544.
[32] A.-S.K. Pathan, H.-W. Lee, C.S. Hong, Security in wireless sensor networks: issues and challenges, in: 2006 8th International Conference Advanced Communication Technology, Vol. 2, IEEE, 2006, pp. 6–pp.
[33] L. Da Xu, W. He, S. Li, Internet of things in industries: A survey, IEEE Trans. Ind. Inform. 10 (4) (2014) 2233–2243.
[34] S. Li, L.D. Xu, S. Zhao, The internet of things: a survey, Inf. Syst. Front. 17 (2015) 243–259.
[35] K.M. Sadique, R. Rahmani, P. Johannesson, Towards security on internet of things: applications and challenges in technology, Procedia Comput. Sci. 141 (2018) 199–206.
[36] L. Tan, N. Wang, Future internet: The internet of things, in: 2010 3rd International Conference on Advanced Computer Theory and Engineering, ICACTE, Vol. 5, IEEE, 2010, pp. V5–376.
[37] B.B. Gupta, M. Quamara, An overview of Internet of Things (IoT): Architectural aspects, challenges, and protocols, Concurr. Comput.: Pr. Exp. 32 (21) (2020) e4946.
[38] P. Asghari, A.M. Rahmani, H.H.S. Javadi, Internet of Things applications: A systematic review, Comput. Netw. 148 (2019) 241–261.
[39] X. Li, R. Lu, X. Liang, X. Shen, J. Chen, X. Lin, Smart community: an internet of things application, IEEE Commun. Mag. 49 (11) (2011) 68–75.

[40] Y. Yuehong, Y. Zeng, X. Chen, Y. Fan, The internet of things in healthcare: An overview, J. Ind. Inf. Integr. 1 (2016) 3–13.
[41] L. Li, Y. Fan, M. Tse, K.-Y. Lin, A review of applications in federated learning, Comput. Ind. Eng. 149 (2020) 106854.
[42] D.C. Nguyen, M. Ding, P.N. Pathirana, A. Seneviratne, J. Li, H.V. Poor, Federated learning for internet of things: A comprehensive survey, IEEE Commun. Surv. Tutor. 23 (3) (2021) 1622–1658.
[43] S. Banabilah, M. Aloqaily, E. Alsayed, N. Malik, Y. Jararweh, Federated learning review: Fundamentals, enabling technologies, and future applications, Inf. Process. Manage. 59 (6) (2022) 103061.
[44] L.U. Khan, W. Saad, Z. Han, E. Hossain, C.S. Hong, Federated learning for internet of things: Recent advances, taxonomy, and open challenges, IEEE Commun. Surv. Tutor. 23 (3) (2021) 1759–1799.
[45] J. Liu, J. Huang, Y. Zhou, X. Li, S. Ji, H. Xiong, D. Dou, From distributed machine learning to federated learning: A survey, Knowl. Inf. Syst. 64 (4) (2022) 885–917.
[46] S. Niknam, H.S. Dhillon, J.H. Reed, Federated learning for wireless communications: Motivation, opportunities, and challenges, IEEE Commun. Mag. 58 (6) (2020) 46–51.
[47] T.N. Rincy, R. Gupta, Ensemble learning techniques and its efficiency in machine learning: A survey, in: 2nd International Conference on Data, Engineering and Applications, IDEA, IEEE, 2020, pp. 1–6.
[48] I.D. Mienye, Y. Sun, A survey of ensemble learning: Concepts, algorithms, applications, and prospects, IEEE Access 10 (2022) 99129–99149.
[49] R. Jurdak, X.R. Wang, O. Obst, P. Valencia, Wireless sensor network anomalies: Diagnosis and detection strategies, in: Intelligence-Based Systems Engineering, Springer, 2011, pp. 309–325.
[50] M.A. Alsheikh, S. Lin, D. Niyato, H.-P. Tan, Machine learning in wireless sensor networks: Algorithms, strategies, and applications, IEEE Commun. Surv. Tutor. 16 (4) (2014) 1996–2018.
[51] M. Mamdouh, M.A. Elrukhsi, A. Khattab, Securing the internet of things and wireless sensor networks via machine learning: A survey, in: 2018 International Conference on Computer and Applications, ICCA, IEEE, 2018, pp. 215–218.
[52] S. Ismail, T.T. Khoei, R. Marsh, N. Kaabouch, A comparative study of machine learning models for cyber-attacks detection in wireless sensor networks, in: 2021 IEEE 12th Annual Ubiquitous Computing, Electronics & Mobile Communication Conference, UEMCON, IEEE, 2021, pp. 0313–0318.
[53] V. Gowdhaman, R. Dhanapal, An intrusion detection system for wireless sensor networks using deep neural network, Soft Comput. (2021) 1–9.
[54] S. Otoum, B. Kantarci, H.T. Mouftah, A novel ensemble method for advanced intrusion detection in wireless sensor networks, in: Icc 2020-2020 Ieee International Conference on Communications, Icc, IEEE, 2020, pp. 1–6.
[55] S. Umamaheshwari, S.A. Kumar, S. Sasikala, Towards building robust intrusion detection system in wireless sensor networks using machine learning and feature selection, in: 2021 International Conference on Advancements in Electrical, Electronics, Communication, Computing and Automation, ICAECA, IEEE, 2021, pp. 1–6.
[56] R. Ul Islam, M.S. Hossain, K. Andersson, A novel anomaly detection algorithm for sensor data under uncertainty, Soft Comput. 22 (5) (2018) 1623–1639.
[57] P. Gulganwa, S. Jain, EES-WCA: energy efficient and secure weighted clustering for WSN using machine learning approach, Int. J. Inf. Technol. 14 (1) (2022) 135–144.
[58] Z. Yu, J.J. Tsai, A framework of machine learning based intrusion detection for wireless sensor networks, in: 2008 IEEE International Conference on Sensor Networks, Ubiquitous, and Trustworthy Computing, Sutc 2008, IEEE, 2008, pp. 272–279.
[59] T.M. Booij, I. Chiscop, E. Meeuwissen, N. Moustafa, F.T. Den Hartog, ToN_IoT: The role of heterogeneity and the need for standardization of features and attack types in IoT network intrusion data sets, IEEE Internet Things J. 9 (1) (2021) 485–496.
[60] L. Breiman, Random forests, Mach. Learn. 45 (2001) 5–32.
[61] A. More, D.P. Rana, Review of random forest classification techniques to resolve data imbalance, in: 2017 1st International Conference on Intelligent Systems and Information Management, ICISIM, IEEE, 2017, pp. 72–78.
[62] J.L. Speiser, M.E. Miller, J. Tooze, E. Ip, A comparison of random forest variable selection methods for classification prediction modeling, Expert Syst. Appl. 134 (2019) 93–101.
[63] H. Zhang, Sc-mlids, GitHub Repository, (2024) https://github.com/Hongwei-Z/SC-MLIDS, Accessed: 2025-04-13.
[64] J.B. Awotunde, S.O. Folorunso, A.L. Imoize, J.O. Odunuga, C.-C. Lee, C.-T. Li, D.-T. Do, An ensemble tree-based model for intrusion detection in industrial internet of things networks, Appl. Sci. 13 (4) (2023) 2479.
[65] Y. Alotaibi, M. Ilyas, Ensemble-learning framework for intrusion detection to enhance internet of things' devices security, Sensors 23 (12) (2023) 5568.
[66] G. Guo, X. Pan, H. Liu, F. Li, L. Pei, K. Hu, An IoT intrusion detection system based on TON IoT network dataset, in: 2023 IEEE 13th Annual Computing and Communication Workshop and Conference, CCWC, IEEE, 2023, pp. 0333–0338.
[67] R.A. Elsayed, R.A. Hamada, M.I. Abdalla, S.A. Elsaid, Securing IoT and SDN systems using deep-learning based automatic intrusion detection, Ain Shams Eng. J. 14 (10) (2023) 102211.
[68] K. Kethineni, G. Pradeepini, Intrusion detection in internet of things-based smart farming using hybrid deep learning framework, Clust. Comput. 27 (2) (2024) 1719–1732.
[69] Z. Cao, Z. Zhao, W. Shang, S. Ai, S. Shen, Using the ToN-IoT dataset to develop a new intrusion detection system for industrial IoT devices, Multimedia Tools Appl. (2024) 1–29.
[70] D.S. Mary, L.J.S. Dhas, A. Deepa, M.A. Chaurasia, C.J.J. Sheela, Network intrusion detection: An optimized deep learning approach using big data analytics, Expert Syst. Appl. 251 (2024) 123919.
[71] Q. Jiao, L. Mhamdi, Deep learning based intrusion detection for IoT networks, in: 2024 Global Information Infrastructure and Networking Symposium, GIIS, IEEE, 2024, pp. 1–6.
[72] I. Almomani, B. Al-Kasasbeh, M. Al-Akhras, WSN-DS: a dataset for intrusion detection systems in wireless sensor networks, J. Sensors 2016 (1) (2016) 4731953.
[73] K.M. Dr Karthick Raghunath, K.S. Dr Arvind, SensorNetGuard: A dataset for identifying malicious sensor nodes, 2023.
[74] S. Suthaharan, M. Alzahrani, S. Rajasegarar, C. Leckie, M. Palaniswami, Labelled data collection for anomaly detection in wireless sensor networks, in: 2010 Sixth International Conference on Intelligent Sensors, Sensor Networks and Information Processing, IEEE, 2010, pp. 269–274.
[75] M. Dener, C. Okur, S. Al, A. Orman, Wsn-bfsf: A new dataset for attacks detection in wireless sensor networks, IEEE Internet Things J. (2023).

Hongwei Zhang received the Bachelor of Computer Science with Honors in 2022 and the Master of Computer Science in 2024 from Dalhousie University, Halifax, Canada. He is currently pursuing his Ph.D. in Computer Science at the same institution. During his master's program, he developed a two-layer intrusion detection system framework for wireless sensor networks based on hybrid machine learning techniques. He has worked as a Research Assistant in the Emerging Wireless Technologies Lab at Dalhousie University, where his research has focused on network security issues in Wireless Sensor Networks, Internet of Things, and Blockchain technologies.

Darshana Upadhyay holds an adjunct faculty position at Dalhousie University where she co-supervises both graduate and undergraduate students in Dr. Sampalli's MYTech lab. She received her Ph.D. degree from the Faculty of Computer Science at Dalhousie University, Canada. She holds a master's degree in computer science from Nirma University, India, where she also served as an assistant professor. During her graduate studies, she was awarded the Gold Medal for securing the first position. Darshana's primary research focuses on network and information security, algorithm conceptualization, hardware design in the field of embedded systems, vulnerability assessments, and intrusion detection and prevention techniques for IoT/SCADA-based systems. She is the co-recipient of the Indo-Canadian Shastri research grant in the field of secure mobile communication. Additionally, she was selected as one of Canada's 2020 Emerging Thought Leaders by the Women in International Security - CANADA. Furthermore, she has received the 2021 Citizenship Award and the 2022 Leadership Award from the Faculty of Computer Science at Dalhousie University.

Marzia Zaman received the M.Sc. and Ph.D. degrees in electrical and computer engineering from the Memorial University of Newfoundland, St. John's, NL, Canada, in 1993 and 1996, respectively. In 1990, she joined the Nortel Networks, Ottawa, ON, Canada, where she joined the Software Engineering Analysis Lab and later joined the Optera Packet Core project as a Software Developer. In addition, she has many years of industry experience as a Researcher and Software Designer with Accelight Networks, Excelocity, Sanstream Technology, and Cistel Technology, Ottawa, ON, Canada. Since 2009, she has been with the Centre for Energy and Power Electronics Research, Queen's University, Canada and one of its industry collaborators, Cistel Technology, on multiple power engineering projects. Her research interests include renewable energy, wireless communication, IoT, cyber security, machine learning, and software engineering.

Achin Jain is a highly accomplished professional with over two decades of experience in the field of computer science and engineering. Holding a Bachelor of Engineering in Computer Science and Engineering from Dr. B.R. Ambedkar University, India, and a Master of Science in Computer Science from the University of Ottawa, Canada, Achin has built a strong foundation in both theory and practice. His career journey includes significant roles such as senior software engineer at Norleaf Networks and Nortel, where he specialized in optical networks, and principal architect at Ciena, where he led initiatives in optical networking technologies. Later, as a technical leader at Cisco, Achin made substantial contributions in layer 2/3 technologies like BGP, MPLS, and Segment Routing. Currently, as a senior research scientist at Norleaf Networks, he continues to leverage his extensive knowledge and expertise in areas such as ad hoc routing, intrusion detection systems, machine learning, and artificial intelligence, contributing to the forefront of network innovation. Achin's research interests lie in exploring innovative solutions in network routing, security, and the application of machine learning and artificial intelligence in network optimization and threat detection.

Srinivas Sampalli (Member, IEEE) received the Bachelor of Engineering degree from Bangalore University, Bangalore, India and the Ph.D. degree from the Indian Institute of Science, Bangalore, India, and is currently a Professor and National 3M Teaching Fellow in the Faculty of Computer Science, Dalhousie University, Halifax, NS, Canada. He has led numerous industry-driven research projects on Internet of Things, wireless security, vulnerability analysis, intrusion detection and prevention, and applications of emerging wireless technologies in healthcare. He currently oversees and runs the Emerging Wireless Technologies Lab and has supervised over 150 graduate students in his career. His primary joy is in inspiring and motivating students with his enthusiastic teaching. He is the recipient of the Dalhousie Faculty of Science Teaching Excellence Award, the Dalhousie Alumni Association Teaching Award, the Association of Atlantic Universities' Distinguished Teacher Award, a teaching award instituted in his name by the students within his Faculty, and the 3M National Teaching Fellowship, Canada's most prestigious teaching acknowledgment. Since September 2016, he holds the honorary position of the Vice President (Canada), of the International Federation of National Teaching Fellows, a Consortium of National Teaching Award winners from around the world.